Coupled-Task Scheduling on Paths, Trees and Beyond: filling the gaps of DP with Knapsack

Niranka Banerjee
Mie University, Japan

Praneet Kumar Patra
Indian Institute of Science Education and Research, Pune, India

Abhishek Sahu
National Institute of Science, Education and Research, an occ of HBNI, India

Abstract

We study single-machine coupled-task scheduling (CTS) in the exact-gap model. Each job i is specified by a triple (A_i, L_i, B_i): it consists of two processing blocks of lengths A_i and B_i separated by an idle gap of fixed length L_i. The objective is to minimize the makespan $C_{\max}$. Compatibility constraints are given by a graph $G = (V, E)$: a job j may be executed inside the idle gap of a job i only if $\{i, j\} \in E$.

Bessy and Giroudeau [6] observed that the complexity of bounded CTS when G is a tree was open, and proved that CTS becomes NP-complete even when G is a star if task lengths are not bounded (via a reduction from Subset Sum). We resolve the tree case in the bounded-length regime by giving an algorithm with running time $\mathcal{O}(n^4 L_{\max}^{10})$, where $L_{\max} = \max_i L_i$, and hence polynomial whenever $L_{\max}$ is polynomially bounded.

For paths, we design a purely combinatorial $\mathcal{O}(n^3)$-time algorithm that works without any boundedness assumption on the gap lengths, and we extend it to all graphs of maximum degree 2. Together with the known NP-hardness of CTS on graphs of maximum degree 3 [13, 19], this yields a sharp degree threshold: CTS is solvable in strongly polynomial time for maximum degree 2, whereas for maximum degree 3 and above it is strongly NP-hard.

Our algorithmic approach on trees is a bottom-up dynamic program that integrates a 0–1 knapsack subroutine as the crucial local decision step. To the best of our knowledge, this is the first use of knapsack as a subroutine inside a dynamic program for coupled-task scheduling on trees, and it may be useful for other scheduling problems with graph-structured compatibility constraints.

2012 ACM Subject Classification Theory of computation → Dynamic programming; Theory of computation → Scheduling algorithms

Keywords and phrases Coupled-task scheduling, compatibility graph, dynamic programming, knapsack, strongly polynomial algorithm

Digital Object Identifier 10.4230/LIPIcs...

© Niranka Banerjee, Praneet Kumar Patra, and Abhishek Sahu;
licensed under Creative Commons License CC-BY 4.0

1 Introduction

Many real-world processes are naturally modeled as two (or more) separated operations with a mandatory waiting period between them: a paint layer must dry before a second coating, a sensor must wait for a physical process to finish before transmitting its readout, a robotic cell must allow an adhesive to cure before continuing assembly and certain medical or maintenance jobs have fixed access windows between subtasks [8, 9, 16, 18]. In such settings, the idle period of a job is not merely wasted time but an opportunity to run other *compatible* jobs, thereby reducing the makespan $C_{\max}$ (the finishing time of the last job). The coupled-task scheduling (CTS) model captures this situation: each job consists of a first processing block, an exact idle gap (the standard exact-gap model [7, 15, 17, 18]), and a second processing block, and compatibility constraints specify which jobs (either completely or partially) may be executed inside another job's gap. Early motivations include interleaving constraints arising in phased-array radar scheduling and related exact-delay models, which also initiated early complexity investigations of the exact-gap setting [16, 17, 18]. The model has been studied across multiple environments: single-machine settings (see the recent survey [15]), parallel identical machines [14] and flow-shop variants with exact time lags [3, 21]; see also the survey [15] for time-dependent and resource-aware formulations and further modeling directions.

Our Problem. We study non-preemptive single-machine scheduling of n *coupled* jobs. Each job $i \in [n]$ is specified by a triple (A_i, L_i, B_i): the machine processes a first block of length A_i (that starts at S_i time), then the job waits *exactly* L_i time units (which we denote as the idle period I_i of the job i), and finally a second block of length B_i is processed (which ends at the time E_i). We refer to this as the exact-gap CTS model. A central feature of the model is the presence of compatibility constraints, represented by an undirected graph $G = (V, E)$ on the job set $V = [n]$. A block of job j, or even its entire execution, may be scheduled within the idle window of job i only if $\{i, j\} \in E$; moreover, if both blocks of j lie inside this window, then the entire triple (A_j, L_j, B_j) must fit within it. Compatibility graphs thus explicitly govern which jobs may overlap and play a fundamental role in shaping feasible schedules.

Coupled-Task Scheduling (CTS)

Input: A set of jobs $V = [n]$ with parameters $\{(A_i, L_i, B_i)\}_{i=1}^{n}$, and an undirected compatibility graph $G = (V, E)$.

Feasible schedule: Start/finish times for all blocks such that

- for each i, the second block of i starts exactly L_i time units after its first block finishes (no preemption of blocks);
- at most one block is processed on the machine at any time;
- if any processing block of job j is executed during the idle window of job i, then $\{i, j\} \in E$; if both blocks of j are executed inside i's idle window, then the entire triple (A_j, L_j, B_j) lies inside.

Objective: Find a feasible schedule minimizing the makespan $C_{\max} = \max_{i \in [n]} E_i$.

Existing Literature. Coupled-task scheduling (CTS) with exact idle gaps has been studied for several decades, with comprehensive surveys consolidating models and baseline

results across single-machine, flow-shop/open-shop, and parallel-machine settings [7, 15]. These surveys document both early complexity boundaries (including strong NP-hardness for various objective/structure choices) and a range of algorithmic techniques such as pseudo-polynomial dynamic programs (DPs), integer programming (IP/CP) formulations, and approximation schemes tailored to restricted parameter regimes.

A substantial line of work incorporates *compatibility constraints* via a graph on the jobs, permitting a job's subtask(s) to run inside another job's idle gap only along edges of the graph. This lens makes the role of *topology* explicit. Simonin and coauthors initiated a systematic study of compatibility graphs, delineating polynomial and NP-hard frontiers as one varies structural assumptions (e.g., stretched/regular tasks, and specific graph families) [20]. Follow-up papers refined these boundaries and gave approximations for several canonical topologies (stars, chains, restricted trees, and certain bipartite classes), showing that both adding and removing edges can tip the balance between tractability and hardness [6, 11]. In parallel, there is a parameterized-complexity strand of literature: for example, Bessy and Giroudeau [6] analyzes FPT algorithms and hardness with respect to natural parameters (e.g., vertex cover of the compatibility graph, bounded processing/idle times), clarifying which structural measures buy algorithmic leverage.

On the algorithmic side, two design patterns recur. *(i) DP-centric methods.* Many exact algorithms are pseudo-polynomial and hinge on dynamic programming over bounded-integer quantities (processing lengths, exact gaps, or derived time budgets). A common framework couples DP with a binary search on the makespan (or on a Lagrange-like threshold), using the DP as a feasibility oracle [15]. Such DPs typically exploit laminarity or acyclicity in precedence/compatibility structure, but their state spaces become prohibitive when local subset choices interact combinatorially. *(ii) Knapsack/packing–centric methods.* Because filling idle gaps is a packing task, a number of approximation algorithms and heuristics reduce local choices to knapsack-like subproblems, sometimes with scaling or FPTAS ideas, and sometimes embedded in IP/CP or greedy/list-scheduling frameworks [20]. These approaches are effective for building high-quality schedules or bounds, yet—when used alone—they typically provide approximations or heuristics rather than exact polynomial algorithms.

Beyond these two patterns, exact MIP/CP formulations (with valid inequalities and specialized branching) and problem-specific greedy/first-fit procedures have proven competitive on moderate-size instances or special topologies [15]. For hardness results, even under strong uniformity assumptions (e.g., identical (A_i, L_i, B_i)) or sparse/dense graph families, the problem can encode classical packing and path problems, yielding strong NP-hardness and inapproximability results [12, 20].

Despite this extensive body of work, the precise interplay between graph topology and numeric parameters in exact-gap CTS remains only partially understood, particularly for fundamental graph classes such as trees and paths. Clarifying these boundaries is important both for theory and for applications, where compatibility constraints often arise from physical layouts, safety considerations, or precedence relations that naturally induce sparse graph structures. This motivates our contribution.

1.1 Our Contribution.

We contribute to the existing literature on CTS problems by providing a landscape of polynomial-time solvability versus hardness. We characterize the problem for tree and path compatibility graphs under both unary and binary encodings, and also determine the complexity of the problem with respect to the maximum degree of the compatibility graph.

A central theme connecting our results is how the complexity differs depending on

the encoding (see Tables 1 and 2). Whenever we present a polynomial-time algorithm for instances with binary encoding, the result also applies to instances with unbounded processing times; accordingly, throughout the paper (in particular in the sections on path and maximum-degree-2 compatibility graphs) we use the terms *binary encoding*, *unbounded durations*, and *superpolynomial durations* interchangeably. Similarly, since under unary encoding the processing times are bounded by the input size, we use the terms *unary encoding*, *polynomially bounded* and *bounded durations* interchangeably.

To the best of our knowledge, no prior work has given an exact algorithm for general trees with arbitrary (A_i, L_i, B_i) optimizing the makespan. In fact, the existence of such an algorithm was stated as an open question by Bessy and Giroudeau (see [6]), who noted that "***it is unknown whether CTS is polynomially solvable when*** G_c ***is a tree***" (Concluding remarks and open problems section of the same paper) and "***Nevertheless the status of CTS problem with*** G_c ***being a tree remains unknown***" (Related works section), where G_c is their compatibility graph. Our algorithmic results for bounded idle gaps resolve this question. Complemented by hardness results of the same paper ([6]) where they proved that CTS becomes NP-complete even when G is a star if task lengths are *n*ot bounded (via a reduction from Subset Sum) we now have a complete picture of CTS on trees.

Let $L_{max} = max\{L_i\}_{i=1}^{n}$. In a unary-encoding L_{max} is polynomially bounded by n, while in binary-encoding it is exponentially bounded by n. We show the following,

▶ **Theorem 1.** *Given a tree compatibility graph with n jobs, CTS can be solved in time* $\mathcal{O}(n^4 L_{\max}^{10})$.

In particular, this yields a polynomial-time algorithm whenever $L_{\max}$ is polynomially bounded in n. A natural next question would be what happens if the compatibility graph is a path? Interestingly, we provide a strict dichotomy between the complexity landscape for CTS on trees and paths, highlighting a sharp contrast between these two graph classes, as summarized in Table 1. In particular, whereas we saw that CTS on trees is polynomial-time solvable only for (polynomially bounded) idle times, CTS on paths admits a strongly polynomial-time algorithm even with binary-encoded lengths! We show the following,

▶ **Theorem 2.** *Given a path compatibility graph with n jobs, CTS can be solved in time* $\mathcal{O}(n^3)$.

Taken together, our results yield a complete algorithmic picture for CTS on path and tree compatibility graphs under both unary (polynomially bounded) and binary (unbounded) encodings of idle lengths.

Table 1 Tractability when compatibility graph is a tree or a path

	Tree	**Path**
Unary encoded lengths ($\leq$ poly(n))	Poly-time (Thm. 1)	Poly-time (Thm. 2)
Binary encoded lengths ($\leq$ exp(n))	NP-hard [6]	Poly-time (Thm. 2)

Next, we move on to clarifying the role of maximum degree of compatibility graphs on the tractability of the problem. By combining the results due to Garey et al. [13] and Simonin et al. [19], one can show that CTS is NP-hard on graphs of maximum degree at least 3 even under unary-encodings.

▶ **Proposition 3.** *CTS is strongly NP hard even when the compatibility graph has maximum degree 3.*

Proof Sketch- Simonin et al. [19] presented a reduction from Hamiltonian Path to CTS in which the compatibility graph of the constructed instance is identical to the input graph. In their construction, each job has total processing time at most 4 and idle time 2. Since Hamiltonian Path is NP-hard even on graphs of maximum degree 3 [13], this immediately implies the proposition. ◀

We complement Proposition 3 by extending the result of Theorem 2 to establish polynomial-time solvability for graphs of maximum degree at most 2. This, thus identifies a tight complexity dichotomy of the problem w.r.t the maximum degree of G as well. Please refer to Table 2 for a summary of results. In particular we show,

▶ **Theorem 4.** *Given a compatibility graph with maximum degree at most* 2*, CTS can be solved in time* $\mathcal{O}(n^3)$*.*

Table 2 Tractability under max-degree of compatibility graphs

	$\Delta(G) \leq 2$	$\Delta(G) \geq 3$
Unary and Binary encoded lengths	Poly-time (Thm. 4)	NP-hard [13, 19]

Contribution to the Algorithmic Landscape. So far, the algorithms proposed in the literature for scheduling problems have typically relied either on dynamic programming or on knapsack based techniques (e.g., [2, 4, 5, 1]). To the best of our knowledge, this is the first algorithm that crucially combines both approaches in order to obtain a polynomial time algorithm.

At a high level, our method integrates a knapsack subroutine within a dynamic programming framework. While a straightforward DP formulation leads to an exponential number of choices at each subtree, making the algorithm inherently exponential, the knapsack component allows us to prune and aggregate these choices efficiently. As a result, the number of relevant configurations can be reduced to a quantity that is polynomial in n.

An overview of this approach is provided in Section 3, with full technical details deferred to the technical section 5. We believe that this strategy is broadly applicable and can be adapted to a range of related scheduling problems, potentially improving running time dependence from exponential to polynomial in n.

1.2 Organization of The Paper

In Section 3, we provide an overview of our algorithms (with a quite detailed overview of our polynomial algorithm on tree compatibility graphs for unary encoding). Section 2 introduces some notations that we use throughout the paper. Section 5 presents our knapsack-based dynamic programming algorithm for CTS on tree compatibility graphs. In Section 6, we give our greedy choice based strongly-polynomial-time algorithm for CTS on path compatibility graphs. Section 7 shows that compatibility graphs with maximum degree 2 also admit strongly-polynomial-time algorithms. Finally, Section 4 concludes and outlines future directions for tightening algorithmic bounds and ideas beyond trees.

2 Preliminaries

We define the set $[n] = \{1, 2, \ldots, n\}$. For a tree G_c (the compatibility graph in our main algorithmic setting), let C_i denote the set of children of node i, and let $p(i)$ represent its parent (if any). We also define $L_{\max} = \max_i L_i$, where L_i is the length of job i. All processing and idle times are integers, as is typical in pseudo-polynomial dynamic programming (DP) approaches. Time flows from left to right on a single machine, which can process at most one block at a time. Given a job i, we say that another job j is *fully contained* in i if the entire execution of j, defined by the tuple (A_j, L_j, B_j), fits entirely within i's idle window. We call j a *left (resp. right) partial overlap* of i if exactly one block of j lies in i's idle window, touching its left (resp. right) boundary, while the other block lies outside of it. Since G_c encodes all admissible pairings, at most one child-subtree block can be processed at a time. We use the variable S_j, E_j, a_j, b_j to denote the time at which the job j starts, the time at which the job j ends, the time when the first block of j ends and the time when the second block of j starts respectively, except explicitly stated otherwise.

Knapsack problem. We briefly recall the classical *0–1* KNAPSACK problem, which will serve as a basic subroutine in our algorithms.

KNAPSACK

Input: A set of m items $x_1, \ldots, x_m$, where each item x_k has an integer weight w_k and value v_k, and a knapsack capacity W.
Objective: Find a subset $K \subseteq [m]$ maximizing $\sum_{k \in K} v_k$ subject to $\sum_{k \in K} w_k \leq W$.

The KNAPSACK problem can be solved via a standard dynamic programming algorithm in pseudo-polynomial time $\mathcal{O}(W \cdot m)$ [10].

3 Algorithmic Framework/Overview

In this section we briefly outline the main ideas behind Theorems 1–4; full details appear in Sections 5– 7. Due to space limitations, we present a comprehensive overview exclusively of our polynomial-time algorithm for tree compatibility graphs under unary encodings.

3.1 A Polynomial Algorithm on Trees under unary-encoding

At a high level, our main contribution in Theorem 1 lies precisely at the intersection of the two algorithmic design patterns: dynamic programming and packing-based (knapsack-style) reasoning to exploit idle gaps as efficiently as possible, which essentially helps us to reduce the running time to polynomial. Bridging this gap motivates the first unified DP-with-knapsack framework that we develop, which in turn allows us to resolve the open question posed in [6].

We root the compatibility tree at a *random vertex* and process it bottom up to design a dynamic programming algorithm. As a preliminary step, we establish a *normalization* of any schedule using shifts and rearrangements. This ensures that, instead of searching over all minimum makespan schedules, it suffices to focus on those that are *nice*. That is, there always exists an optimal schedule satisfying the *niceness* property, and our dynamic program finds such a schedule. Informally, this shows that around the idle gap of any job, the local structure can be assumed to be highly constrained. At most one job may partially overlap the gap from the left, at most one from the right, and all jobs entirely contained within the gap can be arranged contiguously between these two partial overlaps.

More precisely, consider any feasible schedule respecting the compatibility constraints. If we restrict our attention to one of the jobs contained in G_c, then the executions within the idle gap of the job can be rearranged to satisfy the following three structural properties (formal proofs are deferred to Section 3).

- **Property 1.** The jobs contained completely inside the idle time of some other job can be all arranged contiguously.

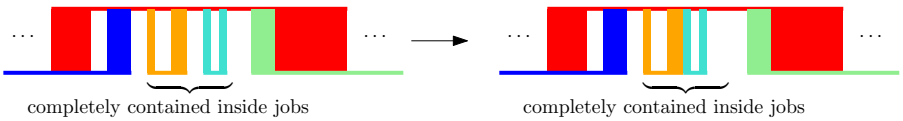


Figure 1 Property 1 - a pictorial rearrangement

- **Property 2.** If the parent job (parent node in the tree G_c) is inside the idle time of one of its child jobs (child node in the tree G_c), it can be safely assumed to be the first such completely contained job.

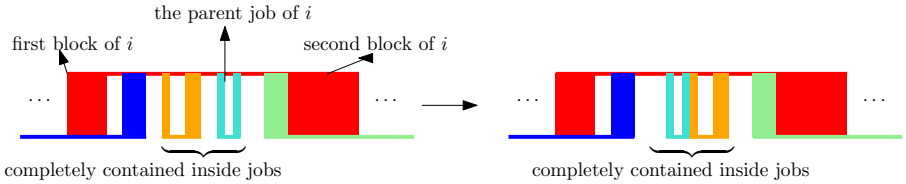


Figure 2 Property 2 - a pictorial rearrangement

- **Property 3.** There is no gap between the last such contained job and the block of the right overlapping job (if one exists). Otherwise, there is no gap between the last such contained job and the right block of the job whose idle time is in consideration.

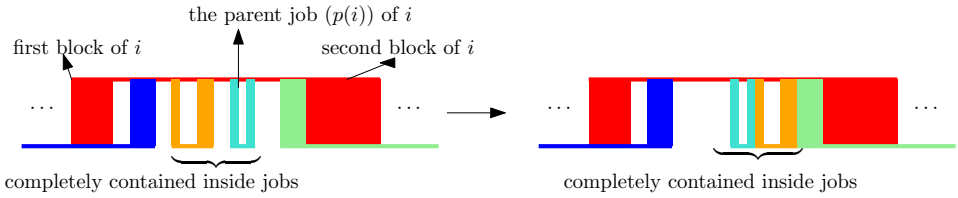


Figure 3 Property 3 - a pictorial rearrangement

Below, we describe the structure of the idle gap of a job in a nice schedule. We formally restate the three properties it satisfies and provide a pictorial illustration, while the formal description can be safely skipped by readers seeking only a high level understanding.

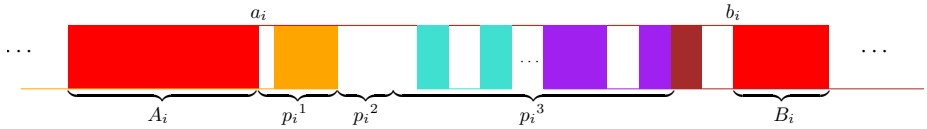


Figure 4 The different time periods and what they mean

An observation using the 3 properties: *Let the compatibility graph G_c be a tree and let S be any feasible schedule. There exists a feasible schedule S^{nice} with the* same makespan *as*

S *such that for every job* i*, the restriction of* S^{nice} *to the idle window* I_i *satisfies all of the following:*

1. *The set of jobs whose* entire triples *(three blocks) are completely contained in* I_i *is the same as in* S*. Moreover, these contained jobs are scheduled* back-to-back *in* I_i *(no machine-idle time between consecutive contained jobs).*
2. *If the parent* $p(i)$ *is completely contained in* I_i*, then* $p(i)$ *is the* first *job in the back-to-back order of contained jobs inside* I_i*.*
3. *The identities of the left and right partial overlap jobs of* i *(if they exist) are the same as in* S*.*
4. *If a right partial overlap of* i *exists, then the last contained job in* I_i *ends exactly at the start time of the overlapping block of that right-overlap job. If no right partial overlap exists, then the last contained job ends exactly at the start time of the second block of* i *(i.e. at the right endpoint of* I_i*).*

We call any schedule S^{nice} a *nice* schedule. Restricting attention to *nice* schedules lets us summarize the interaction between a job's idle interval and the rest of the tree using only 3 parameters (each of which takes values from $[L_{max}]$). Formally, for each job j, we define these parameters (p_j^1, p_j^2, p_j^3) that capture, respectively, how much of the idle gap is occupied by a left partial overlap, how much is reserved (for its parent job) as an internal buffer with no execution, and how much is used to host complete child jobs.

Computing *nice* schedules from *good* sub-schedulings. To compute the makespan of a minimum *nice* schedule, we use a bottom–up dynamic program that first determines the minimum makespan for subtrees of the compatibility graph G_c. A difficulty arises when restricting a global *nice* schedule to a proper subtree: if i is not the root, the induced schedule on the subtree rooted at i may contain gaps irrelevant to the subtree that arise from interactions with jobs outside it.

More precisely, suppose that the parent of i partially overlaps with i through its left block, that is, the left block of the parent is scheduled entirely within the idle time I_i of job i. When the parent of i is removed from the schedule, this may create an unwanted gap between the completion of the last child job contained in i and the start of the second block of i. Such a gap is undesirable when computing the makespan of the restricted schedule.

Observe that this gap can be removed by shifting all child jobs fully contained in i to the right, thereby increasing the buffer period p_i^2. This transformation does not change the overall makespan of the restricted schedule. Moreover, if in a later step of the DP procedure it becomes necessary to place the parent of i inside the idle time of i, these jobs can be shifted back to the left, decreasing the buffer period and creating space for the left block of the parent.

Motivated by this, we define a *good* subschedule as a normalized restricted subtree schedule that removes such irrelevant gaps while preserving the makespan and the flexibility required for higher-level composition in the dynamic program.

Good sub-schedulings. When *nice* schedules are restricted to a subtree, we obtain canonical local forms, which we refer to as good sub-schedulings. This notion captures all the relevant local scheduling behavior within a subtree and allows us to summarize each subtree using a bounded set of parameters. As a result, the dynamic programming algorithm can operate over these good subschedulings without loss of optimality to find a desired *nice* schedule (the final solution).

Fix any feasible nice scheduling S of all jobs, fix a vertex i, and consider the induced sub-schedule S^i consisting of the jobs in the subtree rooted at i (with the same execution

times as in S). By only reordering and translating the jobs that are *completely contained* in the idle time of i, we can obtain another *good* sub-schedule S'^i with the same makespan as S^i that can be combined with $S \setminus S^i$ resulting in a schedule with the exact same makespan as S such that the portion of the schedule inside the idle time of i has the form shown in Figure 5:

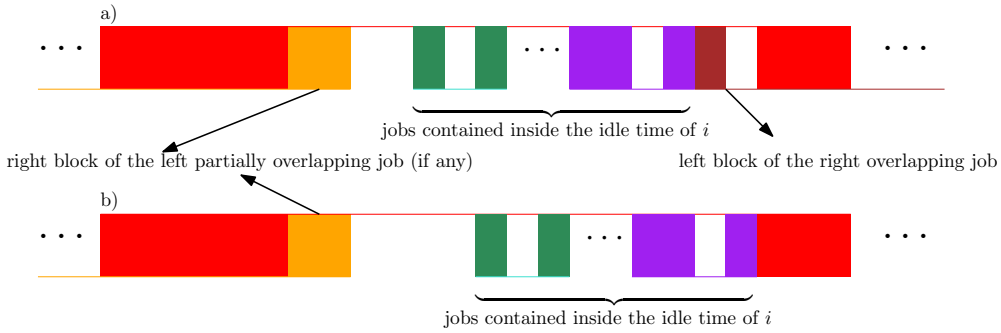


Figure 5 After applying the 3 properties to the sub-schedule ($\mathcal{S}^i$) based on if the right partially overlapping job is present(a) or not(b)

The (possible) left partial overlap block of i appears first, then the contained jobs appear back-to-back with no gaps, and finally the contained block is aligned so that its last job ends exactly when the right partial overlap begins (Figure 5(a)), or at the start of i's second block if there is no right partial overlap (Figure 5(b)). We call any sub-schedule of the subtree rooted at i that has this local form a *good sub-scheduling* (w.r.t. i). We finally show that our DP updates at each node of the tree, remembering minimum makespans of the corresponding subtrees for such *good* sub-schedules (for each possible tuple/triple of (p_i^1, p_i^2, p_i^3)) is enough to construct the minimum makespan *nice* schedule containing all the jobs.

DP States/entries. Formally, our dynamic programming state is $\mathsf{DP}_i(p_i^1, p_i^2, p_i^3)$, which stores the minimum possible makespan of a *good* sub-schedule for all jobs in the subtree rooted at i in G_c, subject to the following conditions:

1. if job i starts at S_i time, a child job partially overlaps the idle gap of i over its first p_i^1 time units;
2. no job other than i (including idle times) is executed during the interval $(S_i + A_i + p_i^1, S_i + A_i + p_i^1 + p_i^2)$ (i.e. the next p_i^2 duration);
3. some child jobs are executed completely within $(S_i + A_i + p_i^1 + p_i^2, S_i + A_i + p_i^1 + p_i^2 + p_i^3)$ (i.e. the next p_i^3 units of time); and
4. a child job partially overlaps the gap over the interval $(S_i + A_i + p_i^1 + p_i^2 + p_i^3, S_i + A_i + L_i)$.

We prove in Section 3 that maintaining the optimal value stored in each DP state suffices to reconstruct a *nice* optimal schedule for the entire instance. In particular, the optimal makespan is obtained by taking the minimum over all DP states associated with the root job. Due to space constraints, we defer the detailed reconstruction procedure to Section 5. There we consider all remaining cases, including those in which the root is contained within the idle time of one of its children, and show that these can be handled directly using the precomputed DP values (for good sub-schedules).

DP entries updates. We consider the following four exhaustive cases (i.e. these four cases exhaustively partition all the possibilities) to compute the required dynamic programming entry. For one of these cases (*Case 3*), we show how the DP entry is computed and also provide an informal argument for its correctness. Job i has,

- **Case 1:** both a left and a right partially-overlapping child;
- **Case 2:** only a right partially-overlapping child;
- **Case 3:** only a left partially-overlapping child;
- **Case 4:** neither left nor right partially-overlapping child.

Though the computation of the DP entry for each of the mentioned cases and argument for their correctness comes in Section 5, we briefly show how a DP entry is updated, just for *Case 3*.

Before showing the transition, for sanity check, we call a tuple\triple (p_i^1, p_i^2, p_i^3) *realizable* if **a)** $p_i^1, p_i^2, p_i^3 \geq 0$, **b)** $p_i^1 + p_i^2 + p_i^3 \leq L_i$ and **c)** $p_i^1 = 0$ ($L_i - p_i^1 - p_i^2 - p_i^3 = 0$) or $p_i^1 > 0$ ($L_i - p_i^1 - p_i^2 - p_i^3 > 0$) provided there is a child job that can partially overlap from the left (right) with the gap defined by p_i^1 ($L_i - p_i^1 - p_i^2 - p_i^3$). (If a tuple is not realizable we define the corresponding DP entry to be $+\infty$.)

Coming to the transition for *Case 3*, let i_ℓ be the left partially-overlapping child with the idle time of the job i and C_i be the set of all the children jobs of i (in the compatibility tree), the DP transition (for a realizable tuple (p_i^1, p_i^2, p_i^3)) is given by

$$\mathsf{DP}_i(p_i^1, p_i^2, p_i^3) = B_i + \min_{i_\ell \in C_i} \min_{(p_{i_\ell}^1, p_{i_\ell}^2, p_{i_\ell}^3)} \left[\mathsf{DP}_{i_\ell}(p_{i_\ell}^1, p_{i_\ell}^2, p_{i_\ell}^3) + \textsc{Pack}_i(\{i_\ell\}, p_i^3) + (p_i^2 + p_i^3) \right], \quad (1)$$

where the inner minimum ranges over all realizable tuples satisfying

$$p_{i_\ell}^2 \geq A_i + p_i^1 - B_{i_\ell}, \qquad p_{i_\ell}^1 + p_{i_\ell}^2 + p_{i_\ell}^3 = L_{i_\ell}, \qquad p_i^1 \geq B_{i_\ell}. \quad (2)$$

Requirement of Constraints (2). The constraints in (2) ensure that there is sufficient space within the idle time of i_ℓ to accommodate a partial overlap of the left block of i and that there is no other partially overlapping job of i_ℓ respectively. In particular, they guarantee that the execution of i_ℓ can be aligned so that its second block overlaps the idle gap of i without violating feasibility by making sure that there is enough space in the buffer portion ($p_{i_\ell}^2$) of the idle time of i_ℓ, such that the jobs entirely contained in $p_{i_\ell}^3$ can be pushed to the left, making space for the first block of i. (figure 6)

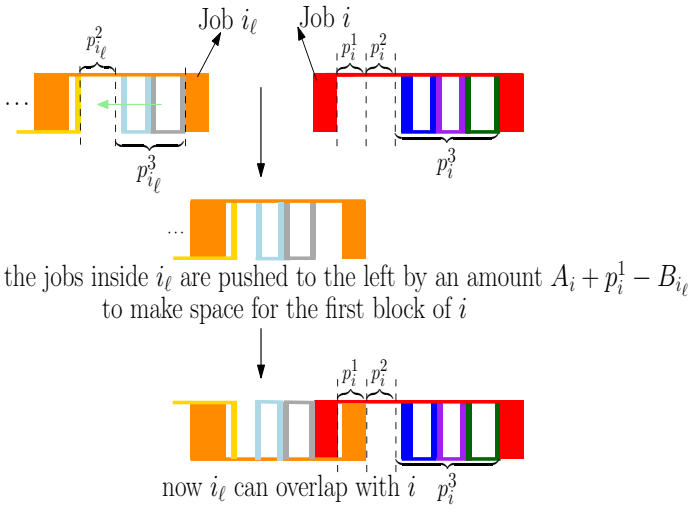


Figure 6 Pictorial explanation for the inequality in 2

Explanation of Eqn. (1). The right-hand side of (1) computes the makespan of a nice schedule for all jobs in the subtree rooted at i in G_c. This is achieved by selecting a child

job i_ℓ whose right block overlaps the idle time of i, together with a compatible choice of parameters $(p^1_{i_\ell}, p^2_{i_\ell}, p^3_{i_\ell})$, and minimizing over all such feasible choices.

Knapsack subroutine to efficiently pack jobs inside. Next, we use crucially a *knapsack subroutine* to compute the term $\textsc{Pack}_i(J', p^3_i)$ (where $J' \subseteq C_i$) efficiently in polynomial time. Without this knapsack-based step, one would have had to go through *exponential* many choices w.r.t. n, preventing the design of a polynomial-time algorithm. Again, the term $\textsc{Pack}_i(J', p^3_i)$ represents the minimum possible makespan contribution *outside* the idle time of job i required to execute all jobs in the subtree T_i of G_c rooted at i, except for the subtrees rooted at the vertices in J'. Intuitively, once we guess the child jobs that partially overlap the idle time of i from the left and from the right, the remaining decision concerns which child subtrees should be embedded entirely inside the idle gap of i, subject to the constraint imposed by p^3_i.

Crucially, for every $j \in J'$, the makespan contribution of the subtree rooted at j is already known from the bottom–up dynamic programming procedure, namely as the value $\mathsf{DP}_j(p^1_j, p^2_j, p^3_j)$. Thus, after fixing the partially overlapping children, the only remaining task is to select a subset of child jobs to place entirely inside the idle time of i so as to minimize the overall makespan of T_i, while respecting the parameters (p^1_i, p^2_i, p^3_i).

The key algorithmic insight underlying our approach is to view this selection problem not as an explicit enumeration over all subsets of child jobs, but instead as a knapsack-type optimization problem. A naive enumeration is exponential in the number of children (and hence exponential in the number n of jobs in the worst case). In contrast, if at each job we can decide which child subtrees to embed inside the idle gap in time polynomial in n and L, this yields a substantial improvement in running time.

Equivalent Knapsack instance formulation. To this end, we observe a close correspondence between our decision problem and a classical 0–1 knapsack instance. At each job i in the bottom–up dynamic programming procedure, the task of selecting which child jobs to place entirely inside the idle time of i is formulated as a knapsack problem. The child jobs are assigned weights and values so that the weight constraint captures which subsets of child jobs can be accommodated inside the idle gap of i, while maximizing the total value corresponds to minimizing the makespan of the subtree rooted at i.

The choice of weights for child jobs is as follows. Since distinct child jobs of i cannot overlap with each other when placed inside the idle time of i (as G_c is acyclic and hence no two children of the same job are adjacent), the weight of a child job i_c is defined as its total duration, including both processing blocks and its idle time. The capacity of the knapsack is given by p^3_i, the portion of the idle gap of i reserved for hosting complete child jobs.

The choice of values of child jobs reflects the reduction in makespan achieved by embedding a child job inside the idle time of i compared to it being outside the idle time of i. Consider a child job i_c of i. If i_c is scheduled outside the idle time of i without any overlap, then, due to the acyclic nature of the compatibility graph, the jobs in the subtree T_{i_c} rooted at i_c cannot overlap with any job in $V(G_c) \setminus V(T_{i_c})$. Consequently, the jobs in T_{i_c} execute independently as a contiguous block of time, yielding a makespan which we denote by $\text{mkspn}_{\text{out}}(i_c)$ (figure 7-(a)).

In contrast, if the job i_c is placed entirely inside the idle time of i, then, by acyclicity, the jobs in $V(T_{i_c}) \setminus \{i_c\}$ cannot overlap with any job in $(V(G_c) \setminus V(T_{i_c})) \cup \{i_c\}$. Moreover, since i_c itself is fully contained within the idle time of i, any overlap with i_c would imply an overlap with i, which is infeasible. Consequently, the jobs in $V(T_{i_c}) \setminus \{i_c\}$ execute contiguously without interference, yielding a minimum possible makespan denoted by $\text{mkspn}_{\text{in}}(i_c)$ (figure **??**-(b)). These minimum possible makespans are known to us due to our bottom–up dynamic

programming procedure: the minimum makespan of $V(T_{i_c}) \setminus \{i_c\}$ is simply the sum of the minimum possible makespans of the children of i_c.

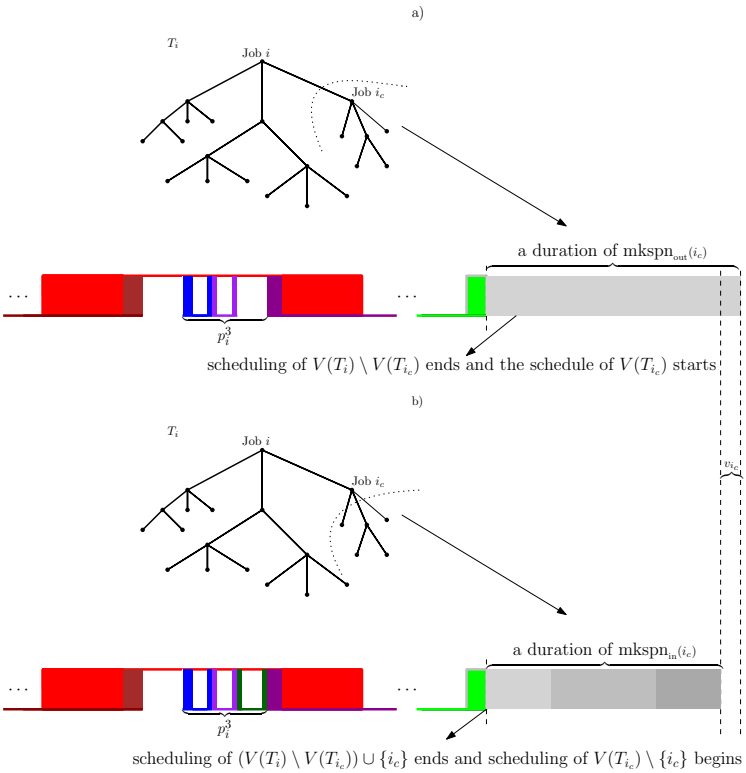


Figure 7 not placing the job $j = i_c$ (a particular child job of i, denoted by dark green) inside the idle time of i (red job), results in the machine being occupied for $\text{mkspn}_{\text{out}}(i_c)$ duration to complete the jobs in $V(T_{i_c})$, while placing it in the idle time results in just $\text{mkspn}_{\text{in}}(i_c)$ occupation of the machine to complete the jobs in $V(T_{i_c}) \setminus \{i_c\}$

We therefore define the value of a child job i_c as (figure 7) $\text{mkspn}_{\text{out}}(i_c) - \text{mkspn}_{\text{in}}(i_c)$, which exactly captures the reduction in makespan obtained by embedding i_c inside the idle time of i versus when it is not contained inside the idle time of i. With these definitions of weights and values, the computation of $\textsc{Pack}_i(J', p_i^3)$ is precisely a 0–1 KNAPSACK problem. The explanations in this section (with a bit more detail as mentioned in section 4) go on to prove our main result-

Theorem 1. *Given a tree compatibility graph with n jobs ($i \in [n]$), CTS can be solved in time $\mathcal{O}(n^4 L_{\max}^{10})$.*

3.2 Strongly-Polynomial-time Algorithms on Paths and Graphs with $\Delta(G) \leq 2$

Bessy and Giroudeau [6] showed that CTS without any boundedness constraint is (weakly) NP-hard even on stars, via a reduction from BIN PACKING/SUBSET SUM. Consequently, pseudo-polynomial algorithms are essentially the best one can hope for on general trees. This naturally raises the question of whether a slightly simpler compatibility structure admits a polynomial-time algorithm, even in the absence of boundedness assumptions. We answer this question in the affirmative by presenting a strongly polynomial-time (or for

unbounded block and idle times) algorithm when the compatibility graph is a path. Below, we provide a brief overview of our approach. Note first that the KNAPSACK-based dynamic programming algorithm we design for tree compatibility graphs also applies to path graphs. However, its running time depends on $L_{\max}$, and therefore it is not strongly polynomial. More generally, any dynamic programming algorithm on a path that explicitly stores partial solutions must remember at least $L_{\max}$ distinct possibilities, which immediately prevents strong polynomiality. To overcome this obstacle, namely, to eliminate the dependence on $L_{\max}$, we develop a greedy approach that avoids storing all partial solutions. We then combine this greedy procedure with a higher-level dynamic program that operates only on the number of jobs, without retaining any information about partial solutions. In this way, we move from a KNAPSACK-based DP to a greedy-choice-based DP, yielding the desired strongly polynomial-time algorithm for path compatibility graphs.

The greedy step. Given a compatibility path graph G_c, any feasible schedule induces an overlap graph H on the same vertex set, where two vertices are adjacent in H if and only if the corresponding jobs overlap in time. Since the schedule respects the compatibility constraints imposed by G_c, the graph H is a subgraph of G_c. When G_c is a path, every connected component of H is itself a contiguous subpath of G_c, corresponding to a sequence of jobs that overlap consecutively on the machine. Leveraging this structural observation, we process each such subpath independently. For a fixed subpath, we first compute the minimum achievable makespan under the assumption that all jobs in the subpath overlap in sequence. This subproblem can be solved using a *greedy* algorithm. The correctness of this greedy argument is deferred to Section 5.

[Lemma 17] In particular, for each job i, we schedule its first block as early as possible within the idle time of the previous job $i-1$, while ensuring that sufficient idle time remains within job i's own window to accommodate the first block of job $i+1$ without overlapping with job $i-1$.

Encapsulating (higher-level) DP. Having computed these values for all subpaths, we perform a second dynamic program that determines an optimal decomposition of G_c into such subpaths, thereby minimizing the overall makespan. Since G_c is a path, this dynamic program proceeds from left to right along the path. At every vertex, the DP stores the minimum makespan achievable by scheduling all jobs up to that vertex. To compute this value, it suffices to consider the starting point of the final subpath ending at the current vertex and combine its precomputed cost with the optimal value for the preceding prefix. With a bit more detail (provided in section 5), we have our second theorem-

Theorem 2. *Given a path compatibility graph with n jobs ($i \in [n]$), CTS can be solved in time $\mathcal{O}(n^3)$.*

A modification of the presented algorithm when $\Delta(G_c) \leq 2$. Finally, when we move beyond trees and paths, we next try to identify the role of the maximum degree of the compatibility graph, yielding tight boundaries in Theorem 4. Graphs with $\Delta(G_c) \leq 2$ decompose into paths, cycles, and isolated vertices. Using a structural lemma for cycles we first show the following.

[Lemma 18] Every cycle of length greater than 3 admits an optimal schedule in which some adjacent pair of jobs does not overlap.

We can delete such an edge and apply our path algorithm component-wise in polynomial time. For cycles of length at most 3, we can simply check all possible overlaps to find an optimal schedule in polynomial time, yielding our desired algorithm. On the other hand,

CTS on graphs with a maximum degree of at least 3 is strongly NP-hard [19], as it reduces from the Hamiltonian path problem, where idle/block times are bounded by 2 and it is well known that Hamiltonian paths even on degree 3 graphs remains NP-hard [13]. Therefore, this gives us a complete classification of the hardness of CTS based on the maximum degree of a compatibility graph, along with the theorem-

Theorem 4. *Given a compatibility graph with maximum degree at most* 2 *with* n *jobs* ($i \in [n]$)*, CTS can be solved in time* $\mathcal{O}(n^3)$.

4 Conclusion

We studied coupled-task scheduling with exact idle gaps under compatibility constraints, focusing on how graph structure governs tractability. Our main positive result gives a pseudo-polynomial dynamic program for tree compatibility graphs that runs in time $\mathcal{O}(n^4 L_{\max}^{10})$ (that, as noted can be improved further), and hence is polynomial-time whenever $L_{\max}$ is polynomially bounded. The core technical ingredient is a bottom–up DP on the tree whose only nontrivial local decision reduces to a 0–1 Knapsack instance over the children: each embedded subtree consumes gap capacity while saving processing time that would otherwise have to appear outside the parent's idle window. This "DP with knapsack" viewpoint cleanly separates local combinatorial choices from global schedule structure.

For path compatibility graphs we obtained a purely combinatorial polynomial-time algorithm running in $\mathcal{O}(n^3)$, independent of gap sizes, and extended it to all graphs of maximum degree 2 (disjoint unions of paths and cycles) with essentially the same order of running time. These results give a complete algorithmic picture for the path/tree regimes under the exact-gap model considered here. Further this goes to give a tight boundary of (strongly) polynomial time solvability with compatibility graph of maximum degree 2, beyond which (for higher degree) the problem is known to become strongly NP-hard.

With the ideas presented in the paper, of using Knapsack to make the crucial decision, it would be interesting to see if fast FPT algorithms would be possible with respect to various parameters that measure closeness of the graph to a tree/forest (such as treewidth).

5 CTS on Trees (explained)

Before describing the dynamic program, we first record a few basic observations about how jobs can overlap in a feasible schedule on a tree compatibility graph. Fix any feasible schedule S (not necessarily optimal), and fix a node i in the compatibility (rooted) tree G_c. Let T_i denote the subtree of G_c rooted at i, and let S^i be the *sub-schedule* induced by S on the jobs in T_i (i.e., we keep the same execution times as in S but ignore all jobs outside T_i).

Our goal is to show that, by only rearranging the jobs that appear inside idle windows (and without changing the total time needed to complete the jobs of T_i), we can transform S^i into another sub-schedule S'^i with a canonical local structure that has the following three properties.

- **Property 1.** The jobs contained completely inside the idle time of some other job can be all arranged contiguously.
- **Property 2.** If the parent job (parent node in the tree G_c) is inside the idle time of one of its child jobs (child node in the tree G_c), it can be safely assumed to be the first such completely contained job.
- **Property 3.** There is no gap between the last such inside job and the block of the right overlapping job (if one exists). Otherwise, there is no gap between the last such inside job and the right block of the job whose idle time is in consideration.

We now formalize these properties as Observations 5–7 below, and then summarize their consequence in Observation 8 by introducing the notion of a *nice schedule*. Note that these transformations/rearrangements preserve feasibility and will not increase the makespan of the sub-schedule as argued later.

▶ **Observation 5.** *Fix a feasible schedule S and a job i. Let I_i denote the idle window of i. Consider the set J_i of jobs whose entire execution (both blocks and their own idle gap) lies inside I_i. Then there exists a feasible schedule S' with the same makespan as S such that, within I_i, the jobs of J_i are scheduled one after another with no time gap between the completion of one job in J_i and the start of the next. (See Figure 8)*

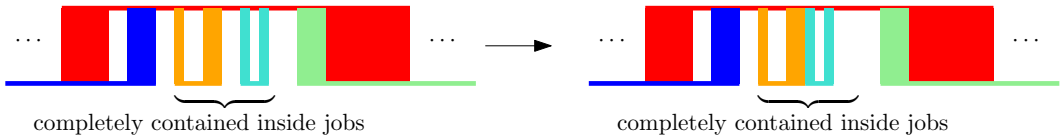


Figure 8 A pictorial description of Observation 5

Proof. If $J_i = \emptyset$ there is nothing to prove. Otherwise, list the jobs of C_i in the order they appear in S within I_i, and denote this order by $i_1, i_2, \ldots, i_m$.

For $k = 1, \ldots, m-1$, let E_{i_k} be the time at which job i_k finishes (end of its second block) in S, and let $S_{i_{k+1}}$ be the start time of the first block of i_{k+1} in S. If $E_{i_k} = S_{i_{k+1}}$ for all k, we are done. Otherwise let k be the smallest index such that $E_{i_k} < S_{i_{k+1}}$, and set $\Delta := S_{i_{k+1}} - E_{i_k} > 0$. We claim that in the original schedule S, the machine is idle throughout the interval $(E_{i_k}, S_{i_{k+1}})$.

Indeed, suppose for contradiction that some block of some job j is processed during $(E_{i_k}, S_{i_{k+1}})$. Since both E_{i_k} and $S_{i_{k+1}}$ lie inside I_i, this implies that a block of j is executed

inside I_i. Because i_k and i_{k+1} are consecutive in the order $i_1, \ldots, i_m$ of *fully contained* jobs, j cannot itself be fully contained in I_i; hence j is a *partial overlap* of i (i.e., exactly one of its two blocks lies inside I_i). In particular, $\{i, j\}$ must be an edge of the compatibility graph. Now consider which block of j lies inside I_i.

Case 1: The *first* block of j lies inside I_i. Then after this first block finishes, job j enters its idle gap of length L_j until its second block starts. Since the second block of j is *not* inside I_i, this idle gap extends up to (and beyond) the right endpoint of I_i. Because i_{k+1} is executed entirely inside I_i and starts after the time when the first block of j is executed (because j has a block in between E_{i_k} and $S_{i_{k+1}}$), at least one block of i_{k+1} is executed during the idle window of j. Therefore $\{j, i_{k+1}\}$ must also be an edge. But we already have $\{i, i_{k+1}\} \in E$ since i_{k+1} is fully contained in I_i. Thus the edges $\{i, j\}, \{i, i_{k+1}\}, \{j, i_{k+1}\}$ form a cycle, contradicting that G_c is a tree.

Case 2: The *second* block of j lies inside I_i. Then j's idle gap ends exactly at the start of this second block, and since the first block of j is not inside I_i, this idle gap covers the left portion of I_i up to the start of j's second block. Because i_k is executed entirely inside I_i and finishes before j's second block begins (as j has a block between E_{i_k} and $S_{i_{k+1}}$), at least one block of i_k is executed during the idle window of j. Hence $\{j, i_k\} \in E$, and together with $\{i, j\} \in E$ and $\{i, i_k\} \in E$ we again obtain a cycle, a contradiction.

In both cases we reach a contradiction. Hence the machine must be idle throughout $(E_{i_k}, S_{i_{k+1}})$.

Therefore we can shift (translate) the entire suffix of jobs $i_{k+1}, i_{k+2}, \ldots, i_m$ earlier by Δ time units (i.e., subtract Δ from the start time of every block of each of these jobs). This does not violate any exact-gap constraint (each shifted job keeps the same internal timing), does not create any machine overlap (we are filling a time interval that was idle), and keeps every shifted job inside I_i (the new start time of i_{k+1} becomes E_{i_k}, which lies in I_i). Moreover, no completion time increases, so the makespan does not increase.

Repeating this operation until no such gap remains yields a schedule S' with the same makespan in which the jobs of C_i occur back-to-back inside I_i, as claimed.

◀

▶ **Observation 6.** *Let S be a feasible schedule and let i be a job. Let J_i be the sequence of jobs whose* entire triples *are scheduled completely inside the idle time of i. Assume that the parent job $p(i)$ (in the tree G_c) also belongs to J_i. Then there exists a feasible schedule S' with the same makespan as S such that, inside the idle time of i, the jobs of J_i are scheduled back-to-back with no gaps (as in Observation 5) and moreover $p(i)$ is the* first *job in this back-to-back order (Figure 9).*

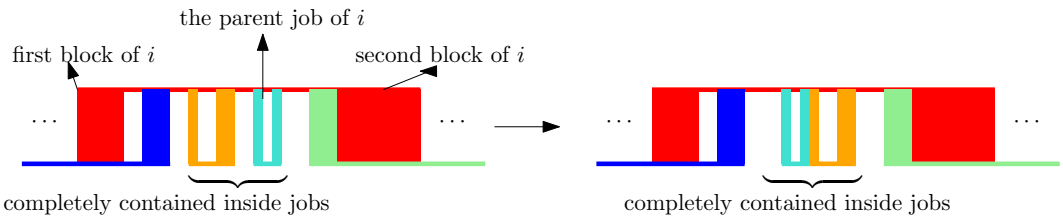


Figure 9 A pictorial description of Observation 6

Proof. Apply Observation 5 to obtain a schedule (still denoted by S) in which the jobs of J_i appear consecutively inside the idle window I_i, with no idle time between the end of one job

in J_i and the start of the next. Let $[t_{\min}, t_{\max}] \subseteq I_i$ be the time interval occupied by these jobs (from the start of the first job of J_i to the completion of the last).

We first note that within $[t_{\min}, t_{\max}]$ the machine executes *only* the triples of the jobs in J_i. Indeed, suppose some job $\ell \notin J_i$ had a processing block scheduled at some time in $[t_{\min}, t_{\max}]$. Then that block lies in I_i, so $\{i, \ell\} \in E(G_c)$. Moreover, since $[t_{\min}, t_{\max}]$ is contained in the union of the triples of jobs in J_i, that block must lie inside the idle window of some job $x \in J_i$, implying $\{x, \ell\} \in E(G_c)$. But also $x \in J_i$ means x is scheduled inside I_i, hence $\{i, x\} \in E(G_c)$. Thus i, x, ℓ form a cycle, contradicting that G_c is a tree. Therefore no such ℓ exists.

Consequently, the restriction of S to $[t_{\min}, t_{\max}]$ is exactly a concatenation of the triples of the jobs in J_i (with no interleaving blocks from other jobs), and the total length of this interval is

$$t_{\max} - t_{\min} \;=\; \textstyle\sum_{x \in J_i} (A_x + L_x + B_x),$$

independent of the order of J_i.

We may therefore permute the jobs of J_i arbitrarily within $[t_{\min}, t_{\max}]$, scheduling their triples back-to-back starting at time $t_{\min}$, while leaving the rest of the schedule unchanged. This preserves feasibility on the single machine (no overlaps are created), preserves every job's exact-gap constraint, keeps every job of J_i inside I_i (since the interval $[t_{\min}, t_{\max}]$ is unchanged and remains within I_i), and does not increase the makespan. Choosing the permutation that places $p(i)$ first yields the desired schedule S'. ◀

▶ **Observation 7.** *Let i be a job, and let S be a feasible schedule. Consider the (possibly empty) sequence of jobs whose triples are completely contained in the idle time of i. Assume this sequence is nonempty. Let j be the job that partially overlaps i from the right (if such a job exists). Then the contained sequence can be arranged so that the* last *contained job ends exactly at the time when the right partially overlapping job j begins. If there is no right partially overlapping job of i, then the last contained job ends exactly at the time when the second block of i begins (see Figure 10).*

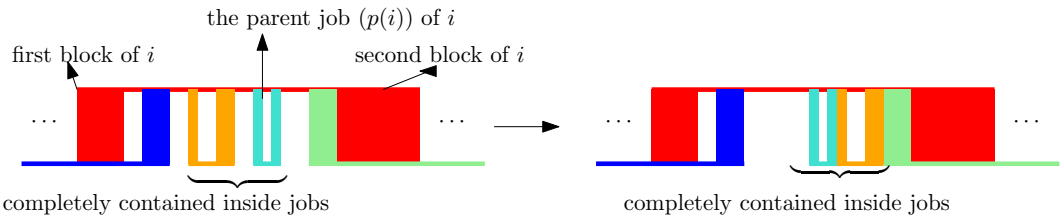


Figure 10 A pictorial description of Observation 7

Proof. Start from the given schedule and apply Observation 6 so that the jobs whose triples are completely contained in the idle time of i are scheduled back-to-back with no gaps between consecutive contained jobs. Let x be the last job in this back-to-back contained sequence, and let E_x denote the completion time of x (i.e., the end of its second block).

If a right partially overlapping job j exists, let S_j be the start time of the block of j that lies in the idle time of i (this is the first block of j by definition of "overlap from the right"). Otherwise, set S_j to be the start time of the second block of i. If $E_x = S_j$, we are done. Assume $E_x < S_j$ and consider the open interval (E_x, S_j). We claim that the machine is idle throughout (E_x, S_j).

Suppose for contradiction that some job ℓ has a processing block executed at some time in (E_x, S_j). Since (E_x, S_j) lies inside the idle time of i, job ℓ has a block inside the idle time of i, hence $\{i, \ell\}$ is an edge of the compatibility graph. Moreover, ℓ cannot be completely contained in the idle time of i (otherwise x would not be the last contained job), so ℓ is a partial overlap of i. There are two cases.

Case 1: The first block of ℓ lies inside the idle time of i (i.e., ℓ overlaps i from the right). In the case when there is no right overlapping job, there cannot be a job ℓ as described as it itself is a right overlapping job, therefore, now we assume there is a right overlapping job j. Then the second block of ℓ lies outside the idle time of i to the right, and therefore the idle window of ℓ covers the entire suffix of the idle time of i after the end of ℓ's first block and up to at least the start of i's second block. In particular, since S_j occurs later than ℓ's first block and still lies inside the idle time of i, the time S_j lies in the idle window of ℓ. Thus a block of j is executed during the idle window of ℓ, which implies $\{\ell, j\}$ is an edge. But we also have $\{i, j\}$ as an edge (since j has a block in the idle time of i), so i, ℓ, j form a cycle, contradicting that G_c is a tree.

Case 2: The second block of ℓ lies inside the idle time of i (i.e., ℓ overlaps i from the left). Then the first block of ℓ lies entirely to the left of the idle time of i, hence the idle window of ℓ starts no later than the beginning of the idle time of i and ends at the start of ℓ's second block. Since x is completely contained in the idle time of i and finishes at time E_x strictly before ℓ's second block begins (because ℓ has a block in (E_x, S_j)), the execution of x occurs during the idle window of ℓ. Therefore $\{\ell, x\}$ is an edge. Together with $\{i, \ell\}$ and $\{i, x\}$ (since x is contained in the idle time of i), we obtain a cycle, again contradicting that G_c is a tree.

In both cases we reach a contradiction. Hence the machine is idle on (E_x, S_j).

Therefore we may shift the entire back-to-back block of contained jobs to the right by $\Delta := S_j - E_x$, without changing the internal structure of any contained job. This preserves feasibility (we shift into a machine-idle interval), preserves all exact-gap constraints (we translate whole triples), keeps all contained jobs inside the idle time of i (the block moves right by exactly the available slack), and does not increase the makespan. After this shift, the last contained job ends exactly at time S_j, as required. ◀

Good sub-scheduling. Combining Observations 5–7, we obtain the following canonical local form. Fix any feasible scheduling S of all jobs, fix a vertex i, and consider the induced sub-schedule S^i consisting of the jobs in the subtree rooted at i (with the same execution times as in S). By only rearranging and shifting the jobs that are *completely contained* in the idle time of i, we can obtain another sub-schedule $S^{i'}$ with the same makespan as S^i such that the portion of the schedule inside the idle time of i has the form shown in Figure 11:

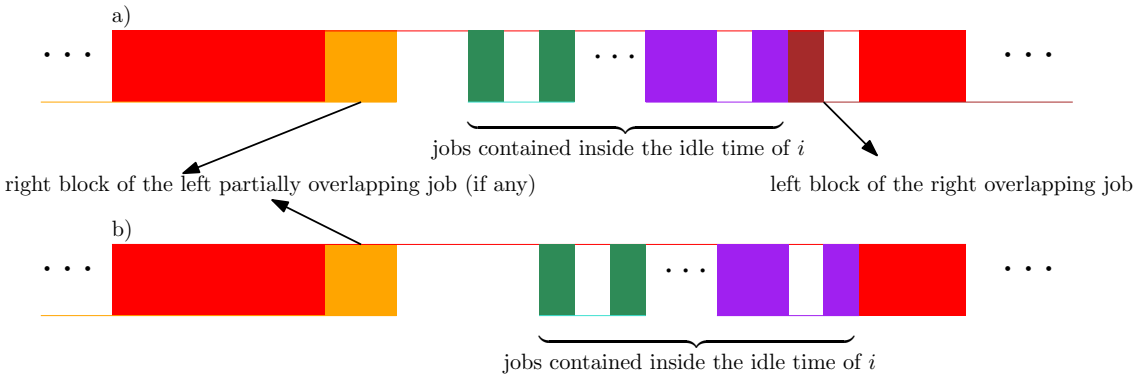


Figure 11 After applying the 3 observations to the sub-schedule $\mathcal{S}^i$ based on if the right partially overlapping job is present(a) or not(b)

- **Case (a): when there is a right partially overlapping job-** the (possible) left partial overlap block of i appears first, then the contained jobs appear back-to-back with no gaps, and finally the contained block is aligned so that its last job ends exactly when the right partial overlap begins (Figure 11(a)), or
- **Case (b): when there is no right partially overlapping job-** at the start of i's second block if there is no right partial overlap (Figure 11(b)). We call any sub-schedule of the subtree rooted at i that has this local form a *good sub-scheduling* (w.r.t. i).

In particular, for a fixed i, the transformations in Observations 5–7 do not change (i) which jobs are fully contained in the idle time of i, nor (ii) which jobs partially overlap i from the left or from the right; they only change the internal arrangement of the contained jobs and their alignment within the idle time of i.

However, one should *not* conclude that there exists an optimal global schedule $S_{\min}$ for which $S^i_{\min}$ is a good sub-scheduling for *every* vertex i simultaneously. Indeed, when the parent of i partially overlaps with i, restricting a global schedule to the subtree of i may delete a block of the parent that lies inside the idle time of i, and the resulting induced sub-schedule need not have the form of Figure 11; see Figure 12 for an example. Another example is when the job i itself is contained in the idle time of one of its children, which is not accounted for in the good schedules.

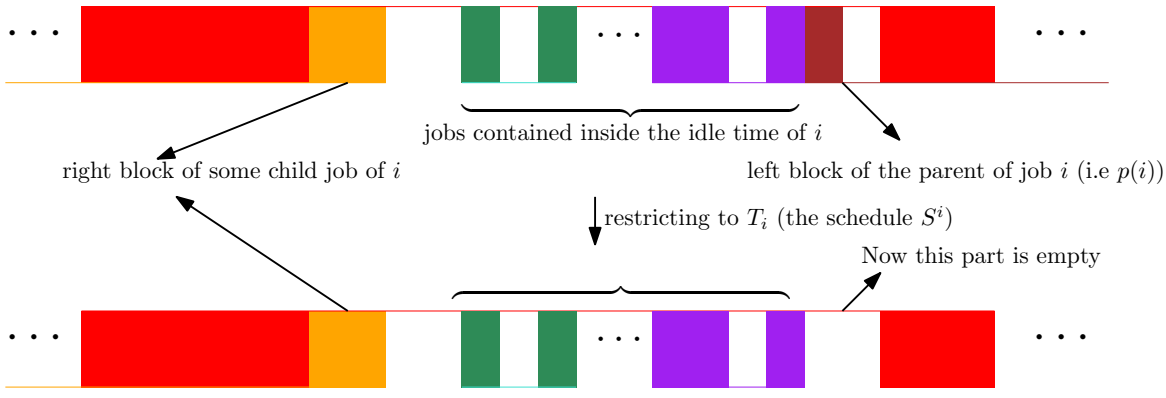


Figure 12 when $\mathcal{S}^i$ is not a good schedule

Nice schedule. However, using these *good* sub-schedules, we can construct a *nice* schedule

containing all the jobs in G_c. Overall, these nice schedules are global counterparts of these local good sub-schedules.

▶ **Observation 8.** *Let G_c be a tree and let S be any feasible schedule. There exists a feasible schedule S^{nice} with the* same makespan *as S such that for every job i, the restriction of S^{nice} to the idle window I_i satisfies all of the following:*

1. *The set of jobs whose* entire triples *are completely contained in I_i is the same as in S. Moreover, these contained jobs are scheduled* back-to-back *in I_i (no machine-idle time between consecutive contained jobs).*
2. *If the parent $p(i)$ is completely contained in I_i, then $p(i)$ is the* first *job in the back-to-back order of contained jobs inside I_i.*
3. *The identities of the left and right partial overlap jobs of i (if they exist) are the same as in S.*
4. *If a right partial overlap of i exists, then the last contained job in I_i ends exactly at the start time of the overlapping block of that right-overlap job. If no right partial overlap exists, then the last contained job ends exactly at the start time of the second block of i (i.e. at the right endpoint of I_i).*

We call any schedule satisfying (1)–(4) for all i, a nice schedule.

Proof. Fix a root of the tree G_c and process the vertices in a postorder (bottom-up) order. We transform S into S^{nice} by applying, for each vertex i when it is processed, Observations 5–7 to the idle window I_i.

More precisely, when processing i, we only perform the operations from Observations 5–7 that: (i) permute the *fully-contained* jobs inside I_i, and/or (ii) translate as a block the consecutive segment formed by these fully-contained jobs *within* I_i, while leaving every job that is not fully contained in I_i unchanged. By Observations 5–7, this enforces properties (1), (2), and (4) for i, and it does not change which jobs are left/right partial overlaps of i, so (3) also holds for i.

It remains to argue that processing i cannot destroy niceness already enforced for any vertex u that was processed earlier in postorder (i.e. any descendant of i). Any modification made while processing i is a permutation/translation of whole triples that are fully contained in I_i. Therefore, there is no change in niceness in the idle time of any of the descendants, u, in particular:

- the order/adjacency relations inside I_u are preserved,
- no new gaps are created inside the back-to-back contained block of u,
- the alignment condition for u (end of last contained job vs. right overlap / start of u's second block) remains true.

Therefore, once a vertex becomes nice, it stays nice throughout the remainder of the procedure. And, after all vertices are processed, the resulting schedule satisfies (1)–(4) for every i, remains feasible, and has the same makespan as S. This is the desired S^{nice}. ◀

A DP algorithm to find the makespan of a optimal nice scheduling. By Observation 8 (existence of a nice schedule), it suffices to compute makespan of an optimal *nice* schedule. Fix an arbitrary root of the compatibility tree G_c (say vertex 1), and for each vertex i let C_i be the set of children of i in this rooted tree. We write T_i for the subtree rooted at i.

Recall that (given a schedule S) for a job i we denote by a_i the end-time of its first block and by b_i the start-time of its second block, so $b_i - a_i = L_i$ (equivalently, the idle window is $I_i = [a_i, b_i]$). A *good* sub-scheduling w.r.t. i has the canonical structure from Figure 11: within I_i there is (i) at most one *left* partially-overlapping neighbor whose *right* block intersects I_i near a_i, (ii) a set of completely contained neighbors scheduled back-to-back, and (iii) at most one *right* partially-overlapping neighbor whose *left* block intersects I_i near b_i.

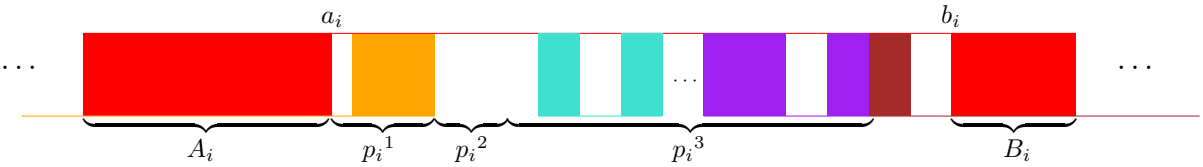


Figure 13 The different time periods and what they mean

Fix a job i. In a nice schedule, the restriction to I_i can be described by three nonnegative integers (p_i^1, p_i^2, p_i^3) (see Figure 13):

- p_i^1 is the length of the initial segment of I_i that is occupied by the right block of the unique left partially-overlapping job of i (if no such job exists then $p_i^1 = 0$);
- p_i^2 is a (possibly empty) slack/buffer segment immediately after p_i^1, during which *no* job from the subtree T_i is processed (this is exactly the "free space" that allows us to later embed the scheduling of i into the idle of a neighbor);
- p_i^3 is the time interval following the slack/buffer during which jobs that are completely contained in I_i are processed back-to-back, with the last such job ending exactly at the end of this period.

The remaining part of the idle window has length $L_i - (p_i^1 + p_i^2 + p_i^3)$, and in a good (hence nice) local form this remaining part is either 0 (no right partially-overlapping job), or it is occupied by the left block of the unique right partially-overlapping job of i.

For sanity check, we call a triple (p_i^1, p_i^2, p_i^3) *realizable* if **a)** $p_i^1, p_i^2, p_i^3 \geq 0$, **b)** $p_i^1 + p_i^2 + p_i^3 \leq L_i$ and **c)** $p_i^1 = 0$ ($L_i - p_i^1 - p_i^2 - p_i^3 = 0$) or $p_i^1 > 0$ ($L_i - p_i^1 - p_i^2 - p_i^3 > 0$) provided there is a child job that can partially overlap from the left (right) with the gap defined by p_i^1 ($L_i - p_i^1 - p_i^2 - p_i^3$). (If a triple is not realizable we define the corresponding DP entry to be $+\infty$.)

DP states/entries. For each job i and each realizable triple (p_i^1, p_i^2, p_i^3), let

$$\mathsf{DP}_i(p_i^1, p_i^2, p_i^3)$$

denote the minimum possible makespan of a good sub-scheduling of all jobs in the subtree T_i *consistent with the local description of* I_i *given by* (p_i^1, p_i^2, p_i^3). For a non-realizable triple we set $\mathsf{DP}_i(p_i^1, p_i^2, p_i^3) = +\infty$. Further, we call the minimum possible makespan of a sub-tree T_i as $\mathbb{DP}(i)$ (formally defined in Equation (13)). We bring it to the reader's attention that DP and $\mathbb{DP}$ are different (functions).

As G_c is a tree, any job that overlaps with i must be adjacent to i in G_c (otherwise the overlap edge together with the unique path would create a cycle). Hence, within the scheduling of T_i, the only jobs that can intersect I_i are the children of i, denoted as elements of the set C_i (possibly in a partially-overlapping manner). This is exactly why the recursion below only branches over children.

Before going into the recurrence relation for our DP entries, we describe the following knapsack-based subroutine, that drastically reduces the time complexity. We then use this in defining the recurrence relations for our DP entries.

A Knapsack-based subroutine. Fix i and suppose we have already decided which (if any) children are the partially-overlapping ones. Let $D \subseteq C_i$ be the set of those designated boundary children (so $D \in \{\emptyset, \{i_\ell\}, \{i_r\}, \{i_\ell, i_r\}\}$). The remaining children $C_i \setminus D$ are the candidates to be *completely contained* inside the p_i^3-period of I_i. Below, we create a KNAPSACK instance where the children (of i) are the items, and the solution to this knapsack gives us the best possible way of packing/scheduling child jobs inside the idle time (I_i) of i.

- *Assigning weights:* For a child $j \in C_i \setminus D$, define its *weight*

$$w_j \;:=\; A_j + L_j + B_j,$$

  which is exactly the amount of time inside I_i consumed if job j (i.e., the whole coupled task of j) is placed completely inside I_i.
- *Assigning values:* For a child $j \in C_i \setminus D$, define its *value*

$$v_j \;:=\; \mathbb{DP}(j) \;-\; \textstyle\sum_{k \in C_j} \mathbb{DP}(k),$$

The choice of values of child jobs reflects the reduction in makespan achieved by embedding a child job inside the idle time of i compared to it being outside the idle time of i. Consider a child job j of i. If j is scheduled outside the idle time of i without any overlap, then, due to the acyclic nature of the compatibility graph, the jobs in the subtree T_j rooted at j cannot overlap with any job in $V(G_c) \setminus V(T_j)$. Consequently, the jobs in T_j execute independently as a contiguous block of time, yielding a makespan which we denote by $\text{mkspn}_{\text{out}}(j) = \mathbb{DP}(j)$.

In contrast, if the job j is placed entirely inside the idle time of i, then, by acyclicity, the jobs in $V(T_j) \setminus \{j\}$ cannot overlap with any job in $(V(G_c) \setminus V(T_j)) \cup \{j\}$. Moreover, since j itself is fully contained within the idle time of i, any overlap with j would imply an overlap with i, which is infeasible. Consequently, the jobs in $V(T_j) \setminus \{j\}$ execute contiguously without interference, yielding a minimum possible makespan denoted by $\text{mkspn}_{\text{in}}(j) = \sum_{k \in C_j} \mathbb{DP}(k)$. These minimum possible makespans are known to us due to our bottom–up dynamic programming procedure: the minimum makespan of $V(T_j) \setminus \{j\}$ is simply the sum of the minimum possible makespans of the children of j.

We therefore define the value of a child job j as (figure 7)

$$\text{mkspn}_{\text{out}}(j) - \text{mkspn}_{\text{in}}(j),$$

which exactly captures the reduction in makespan obtained by embedding j inside the idle time of i versus when it is not contained inside the idle time of i.

$\mathbb{DP}(j)$ is defined below as the best (minimum) makespan for subtree T_j. Intuitively, v_j is the reduction in *time that must be spent outside the execution of i to schedule the jobs in T_j* when we manage to place j itself inside I_i; in that case, the only remaining contribution of T_j that cannot overlap i directly comes from the child-subtrees of j (since in a tree, descendants of j are not adjacent to i).

We capture the optimal choice of completely-contained children via a standard 0–1 knapsack, i.e. the answer to our knapsack instance actually provides us with a set of children jobs whose inclusion into the idle time of job i, gives us the minimum possible makespan for the jobs in $V(T_i) \setminus \{i\}$ that is executed outside i. This is captured by our following definition of $\textsc{Pack}_i(\cdots)$

▶ **Definition 9.** *For $D \subseteq C_i$ and capacity $p \in \{0, 1, \ldots, L_i\}$, define*

$$\textsc{Pack}_i(D, p) := \sum_{j \in C_i \setminus D} \mathbb{DP}(j) - \max\Big\{\sum_{j \in S} v_j : S \subseteq C_i \setminus D,\ \sum_{j \in S} w_j \le p\Big\}. \quad (3)$$

Equivalently, if S is an optimal knapsack solution, then

$$\textsc{Pack}_i(D, p) = \sum_{j \in (C_i \setminus D) \setminus S} \mathbb{DP}(j) + \sum_{j \in S} \sum_{k \in C_j} \mathbb{DP}(k). \quad (4)$$

The correctness of the formula for computing $\textsc{Pack}_i(D, p)$ in Definition 4.1 follows from the lemma below, which additionally proves the equivalence of Equations 3 and 4.

▶ **Lemma 10.** *Fix i, a boundary set $D \subseteq C_i$, and a realizable triple (p_i^1, p_i^2, p_i^3). Among the children in $C_i \setminus D$, the minimum possible contribution to the makespan of T_i that is compatible with placing a subset of them completely inside the p_i^3-period of I_i is exactly $\textsc{Pack}_i(D, p_i^3)$.*

Proof. In a nice schedule, the children in $C_i \setminus D$ that are completely contained in I_i are processed back-to-back inside the p_i^3-period. If we choose a subset $S \subseteq C_i \setminus D$ to be contained, then the total time they occupy inside I_i is $\sum_{j \in S} w_j$, hence feasibility requires $\sum_{j \in S} w_j \le p_i^3$.

If a child $j \notin S$ is not contained in I_i, then its entire subtree T_j must be completed outside this p_i^3-period; the optimal such contribution is $\mathbb{DP}(j)$ by definition.

If a child $j \in S$ is contained in I_i, then only j itself can overlap with i (tree compatibility graph property); consequently, any work in T_j that must occur outside the execution of i can only come from subtrees rooted at children of j. Optimally, this costs $\sum_{k \in C_j} \mathbb{DP}(k)$. Therefore, for a fixed feasible S the resulting cost is

$$\sum_{j \notin S} \mathbb{DP}(j) + \sum_{j \in S} \sum_{k \in C_j} \mathbb{DP}(k) = \sum_{j \in C_i \setminus D} \mathbb{DP}(j) - \sum_{j \in S} \Big(\mathbb{DP}(j) - \sum_{k \in C_j} \mathbb{DP}(k)\Big) = \sum_{j \in C_i \setminus D} \mathbb{DP}(j) - \sum_{j \in S} v_j.$$

Minimizing this cost is therefore equivalent to maximizing $\sum_{j \in S} v_j$ subject to $\sum_{j \in S} w_j \le p_i^3$, which is exactly the knapsack definition of $\textsc{Pack}_i(D, p_i^3)$. ◀

Updating DP entries. We now provide a recurrence relation for $\mathsf{DP}_i(p_i^1, p_i^2, p_i^3)$ by considering the following four exhaustive cases, on whether i has a left partially-overlapping child, a right partially-overlapping child, both, or neither. In a nice schedule these are determined exactly by

$$\text{left overlap exists} \iff p_i^1 > 0, \qquad \text{right overlap exists} \iff p_i^1 + p_i^2 + p_i^3 < L_i.$$

Accounting for A_i and B_i in the makespan. The additive $(p_i^2 + p_i^3)$ term in the DP updates accounts for the portion of I_i *after* the initial left-overlap (if any) whose duration is not accounted for in the DP entries of any of its overlapping children or Pack functions- which only calculate the duration due to the descendant jobs of the child jobs (of i) that are completely contained in the idle time of I_i and the jobs in the subtrees of the children that are completely contained outside I_i.

A_i is added in the duration whenever their is no left overlap child, as, whenever there is a left overlapping child, A_i (and p_i^1) are already accounted for in the DP entries of the left overlapping child. Therefore, if there is no left overlap child, we add an extra amount A_i to

the makespan. Similar argument works for the term B_i, depending whether there is a right overlapping child or their is no such right overlapping child.

Now we move to specify the updates for DP and $\mathbb{DP}$ entries for a job i. Note, as argued in the main body of the paper, we can shift and create space in the idle time of the children jobs that have a good sub scheduling if they have to partially overlap with the job i.

Case 1: both a left and a right partially-overlapping child.

Here $p_i^1 > 0$ and $p_i^1 + p_i^2 + p_i^3 < L_i$. Let i_ℓ be the unique left-overlapping child and i_r the unique right-overlapping child (necessarily distinct).

When the right/second block of i_ℓ and the left/first block of i_r overlaps with the idle time of i:

$$\mathsf{DP}_i(p_i^1,p_i^2,p_i^3) = \min_{\substack{i_\ell,i_r\in C_i\\ i_\ell\neq i_r}} \min_{\substack{(p_{i_\ell}^1,p_{i_\ell}^2,p_{i_\ell}^3)\\(p_{i_r}^1,p_{i_r}^2,p_{i_r}^3)}} \Big[\mathsf{DP}_{i_\ell}(p_{i_\ell}^1,p_{i_\ell}^2,p_{i_\ell}^3)+\mathsf{DP}_{i_r}(p_{i_r}^1,p_{i_r}^2,p_{i_r}^3)+\textsc{Pack}_i(\{i_\ell,i_r\},p_i^3)+(p_i^2+p_i^3)\Big], \tag{5}$$

where the inner minimum ranges over realizable triples satisfying the interface constraints

$$\begin{aligned} p_{i_\ell}^2 \geq A_i + p_i^1 - B_{i_\ell}, \quad p_{i_\ell}^1 + p_{i_\ell}^2 + p_{i_\ell}^3 = L_{i_\ell}, \quad p_{i_r}^1 = 0,\\ p_{i_r}^2 \geq B_i + L_i - (p_i^1+p_i^2+p_i^3) - A_{i_r}, \quad p_i^1 \geq B_{i_\ell}, \quad L_i - (p_i^1+p_i^2+p_i^3) \geq A_{i_r}. \end{aligned} \tag{6}$$

The first constraint essentially says that we can push the jobs inside i_ℓ to the left and make space for the first block of i (figure - 6), similarly the last constraint essentially implies there is enough space in the buffer space of i_r that the last block of i can be placed inside it and that there is enough space in the job i's idle time to accommodate the left and right overlapping jobs (all the while maintaining the good sub scheduling properties).

Case 2: only a right partially-overlapping child.

Here $p_i^1 = 0$ and $p_i^2 + p_i^3 < L_i$. Let i_r be the right-overlapping child.

When the left block of i_r overlaps with the idle time of i:

$$\mathsf{DP}_i(0,p_i^2,p_i^3) = A_i + \min_{i_r\in C_i} \min_{(p_{i_r}^1,p_{i_r}^2,p_{i_r}^3)} \Big[\mathsf{DP}_{i_r}(p_{i_r}^1,p_{i_r}^2,p_{i_r}^3)+\textsc{Pack}_i(\{i_r\},p_i^3)+(p_i^2+p_i^3)\Big], \tag{7}$$

where the inner minimum ranges over realizable triples satisfying

$$p_{i_r}^1 = 0, \quad p_{i_r}^2 \geq B_i + L_i - (p_i^2+p_i^3) - A_{i_r}, \quad L_i - (p_i^1+p_i^2+p_i^3) \geq A_{i_r}. \tag{8}$$

Case 3: only a left partially-overlapping child.

Here $p_i^1 > 0$ and $p_i^1 + p_i^2 + p_i^3 = L_i$ (no right overlap). Let i_ℓ be the left-overlapping child.

When the right block of i_l overlaps with the idle time of i:

$$\mathsf{DP}_i(p_i^1,p_i^2,p_i^3) = B_i + \min_{i_\ell\in C_i} \min_{(p_{i_\ell}^1,p_{i_\ell}^2,p_{i_\ell}^3)} \Big[\mathsf{DP}_{i_\ell}(p_{i_\ell}^1,p_{i_\ell}^2,p_{i_\ell}^3)+\textsc{Pack}_i(\{i_\ell\},p_i^3)+(p_i^2+p_i^3)\Big], \tag{9}$$

where the inner minimum ranges over realizable triples satisfying

$$p_{i_\ell}^2 \geq A_i + p_i^1 - B_{i_\ell}, \quad p_{i_\ell}^1 + p_{i_\ell}^2 + p_{i_\ell}^3 = L_{i_\ell}, \quad p_i^1 \geq B_{i_\ell}. \tag{10}$$

Case 4: neither left nor right partially-overlapping child.

Here $p_i^1 = 0$ and $p_i^2 + p_i^3 = L_i$. Then

$$\mathsf{DP}_i(0, p_i^2, p_i^3) = (A_i + B_i) + \textsc{Pack}_i(\emptyset, p_i^3) + (p_i^2 + p_i^3). \tag{11}$$

▶ **Definition 11.** *Let $\mathbb{DP}_i(\varphi)$ be the minimum makespan of a nice scheduling of T_i in which the entire execution of job i (first block, idle, second block) is placed within the idle segment of a single child $j \in C_i$:*

$$\mathbb{DP}_i(\varphi) := \min_{j \in C_i} \min_{(p_j^1, p_j^2, p_j^3)} \left\{ \mathsf{DP}_j(p_j^1, p_j^2, p_j^3) + \sum_{k \in C_i \setminus \{j\}} \mathbb{DP}(k) \;:\; p_j^2 \geq A_i + L_i + B_i \right\}. \tag{12}$$

(As before, triples that are not realizable for j contribute $+\infty$.)

$\mathbb{DP}(i)$ and correctness

▶ **Definition 12.** *For each node i, define*

$$\mathbb{DP}(i) \;:=\; \min \left\{ \min_{\substack{p_i^1, p_i^2, p_i^3 \geq 0 \\ p_i^1 + p_i^2 + p_i^3 \leq L_i}} \mathsf{DP}_i(p_i^1, p_i^2, p_i^3),\; \mathbb{DP}_i(\varphi) \right\}. \tag{13}$$

▶ **Lemma 13** (Correctness). *Let G_c be a compatibility tree rooted at 1. The value $\mathsf{DP}(1)$ equals the minimum makespan over all feasible schedules (and hence over all nice schedules).*

Proof. Since to compute the value of $\mathbb{DP}(i)$, we are taking the minimum over all the possible $\mathbb{DP}(\varphi)$ entries and the $\mathsf{DP}(p_i^1, p_i^2, p_i^3)$ entries (which is exhaustive as a job can either be entirely contained in another job or not respectively), it suffices to only claim that each of these entries are correctly/optimally computed. To this end, we use induction. It is straightforward to see that all the DP entries (including the $\mathbb{DP}$ entries) for the leaf nodes are correct, in particular, all the feasible DP_l (and $\mathbb{DP}_l$) entries are the same and are all equal to the entire duration of the considered leaf job (l).

Now consider a job k and assume all the descendants of k have their DP (and $\mathbb{DP}$) entries correctly computed (to be the minimum possible makespan of the jobs in the subtree rooted at k following the constraints set by the entries). First consider the $\mathbb{DP}_k$ entry - $\mathbb{DP}_k(\phi)$. To achieve this, we know that one of the child jobs should contain this job. Using Observation 6, we are safe to assume that the job k is the first job in the sequence of jobs that are contained in the idle time of k_c (the child that contains k in its I_{k_c}). Therefore the $\mathbb{DP}$ entry update is correct for $\mathbb{DP}_k(\phi)$ as it essentially takes the minimum makespan over all the cases where k is in the idle time of one of its children (while the other children k'_c subtrees are scheduled as separate chunks with makespans of $\mathbb{DP}(k'_c)$ respectively).

Now we move to the $\mathsf{DP}_k(p_k^3, p_k^3, p_k^3)$. for the leaf nodes, these are always set to ∞ for all the leaf (l) entries where $p_l^1 > 0$ and/or $p_l^1 + p_l^2 + p_l^3 < L_l$, because there are no child jobs that can overlap from the left or the right respectively to fill the gap. The other entries for the root nodes is the same as the duration of the job itself (again, as there are no descendants). Now assume that we have all the entries (for good sub scheduling) correct for the descendants of k. Following the same line of argument as above, the DP entries for the node are correct since we are evaluating it over all the possible left, right overlaps, as well as efficiently filling the p_k^3 period using the $\textsc{Pack}$ function. However, for the sake of completeness, we show the correctness of the DP expression 5, i.e. the case of both left and right overlapping job. Once

we show this, the correctness for the other 3 cases follow (because the other 3 cases either have only one partially overlapping job or no partially overlapping job).

Assume that the minimum functions select the jobs i_ℓ and i_r with the corresponding time stamps $(p^1_{i_\ell}, p^2_{i_\ell}, p^3_{i_\ell})$ and $(p^1_{i_r}, p^2_{i_r}, p^3_{i_r})$. However, for the sake of contradiction, assume that the jobs j_ℓ and j_r (both being children of i) with the corresponding time stamps $(p^1_{j_\ell}, p^2_{j_\ell}, p^3_{j_\ell})$ and $(p^1_{j_r}, p^2_{j_r}, p^3_{j_r})$ (corresponding to good sub-schedules of the respective sub trees) where $p^k_{j_\ell} \leq L_{j_\ell}$ and $p^k_{j_r} \leq L_{j_r}$ result in the minimum configuration. Now the only thing we need to prove is, all these time stamps satisfy the inequations in 6-

1. Since the left partially overlapping job should not have a right partially overlapping child job, we require that $p^1_{j_\ell} + p^2_{j_\ell} + p^3_{j_\ell} = L_{j_\ell}$.
2. Since there should be enough gap in the idle time of the job j_ℓ to shift all the completely contained jobs in j_ℓ towards earlier starting time and create enough space for the first block of i, while itself ending within the first p^1_i after the end of the first block of i, we need that $p^2_{j_\ell} \geq A_i + p^1_i - B_{j_\ell}$
3. Since the second block of the job j_ℓ is to be scheduled right after the completion of the first block of the job i, within the first p^1_i time duration, we require that $p^1_i \geq B_{j_\ell}$.
4. Similarly for the job j_r, there should not be any of its child jobs that overlap with it partially from the left, which required that $p^1_{j_r} = 0$.
5. Again, since j_r overlaps with i partially from the right, there should be enough space in the idle time of the job j_r so that only the second block of i can be completed in the idle time of j_r just after the completion of the first block of j_r all the while maintaining the constraints set by p^k_i for all $k \in [3]$, which mandates - $p^2_{j_r} \geq B_i + L_i - (p^1_i + p^2_i + p^3_i) - A_{j_r}$.
6. Again, since the first block of the job j_r is to be scheduled in the idle time of the job i, just before the start of the second block of the job i, the time stamps should satisfy-$L_i - (p^1_i + p^2_i + p^3_i) \geq A_{j_r}$.

Therefore, even for the choice of j_r and j_ℓ, the inequations in 6 are satisfied, this observation along with lemma 10 essentially means that the minimum functions went over these choices and yet chose the jobs i_r and i_ℓ (with the mentioned choice of time stamps). This shows that our assumption was wrong and indeed the jobs i_r and i_ℓ with the mentioned time stamps result in the least makespan of the jobs in T_i subject to the time stamp- (p^1_i, p^2_i, p^3_i) of the job i

◀

Runtime analysis. For each node i there are at most L^3_i triples (p^1_i, p^2_i, p^3_i) and hence a total of $n \times L^3_{max}$ many DP entries. For a fixed DP entry, evaluating the right-hand side of (5) requires trying all ordered pairs $(i_\ell, i_r) \in C_i \times C_i$ (along with their corresponding triples), going over a total of $n^2 \times \mathcal{O}(L^6_{\max})$ many choices and, for each such choice, running one KNAPSACK instance of capacity at most L_i on at most $|C_i|$ items which has an algorithm running in time $\mathcal{O}(|C_i| \cdot L_i)$ ([10]). Therefore, for case-1, we have an overall runtime of $\mathcal{O}\big(L^9_i \cdot |C_i|^2 \cdot (|C_i| \cdot L_i)\big) = \mathcal{O}\big(|C_i|^3 \cdot L^{10}_i\big)$ time at node i. Cases 2–4 are cheaper by at least a factor $|C_i|$.

For the φ option (12), we try $\mathcal{O}(|C_i|)$ choices for the containing child j and then scan realizable triples for j (at most $L^3_j \leq L^3_{\max}$), leading to $\mathcal{O}(|C_i| \cdot L^3_{\max})$ time per node. Summing over all nodes and using $|C_i| \leq n$ and $L_i \leq L_{\max}$, we obtain a polynomial bound:

$$\textstyle\sum_{i \in V} \mathcal{O}\big(|C_i|^3 \cdot L^{10}_i + |C_i| \cdot L^3_{\max}\big) \;\leq\; \mathcal{O}\big(n^4 \cdot L^{10}_{\max}\big).$$

Thus the algorithm runs in polynomial time in the input size (in particular when $L_{\max}$ is polynomially bounded in n).

Theorem 1. *Given a tree compatibility graph with n jobs ($i \in [n]$); CTS can be solved in time $\mathcal{O}(n^4 L_{\max}^{10})$.*

However, a clever look reveals that we don't need to guess all the possible triples for the left an right overlapping children if we (beforehand) remember their optimal possible makespan for each value of p_{i_ℓ} (or p_{i_r}) conditioned to if there are left and/or right overlapping children of ℓ (r), as the check for realizable triples only checks these. But this is not of much interest as it only reduces the dependence on $L_{\max}$, still keeping it pseudopolynomial.

6 CTS on Paths (explained)

Giroudeau [6] showed that CTS without boundedness constraints is weakly NP-hard even on stars, via a reduction from Bin Packing/Subset Sum. So on general trees, bounding the block and idle times is necessary for a polynomial-time algorithm. This leaves open whether paths, the other simple tree structure, admit one without such bounds.

We answer this affirmatively with a strongly polynomial-time algorithm for path compatibility graphs, with no bounds on block or idle times. The knapsack-based dynamic program for trees also applies to paths, but its dependence on $L_{\max}$ rules out strong polynomiality, since any DP storing partial solutions on a path must track at least $L_{\max}$ possibilities. We instead replace the stored partial solutions with a *greedy* procedure, combined with a higher-level DP that depends only on the number of jobs.

For some notations, each job $i \in [n]$ has parameters (A_i, L_i, B_i) and, under a given schedule, a start time S_i. Its first block occupies $[S_i, S_i{+}A_i]$, its idle window is $I_i = [S_i{+}A_i, S_i{+}A_i{+}L_i]$, and its second block occupies $[S_i{+}A_i{+}L_i,\ E_i]$, where $E_i = S_i{+}A_i{+}L_i{+}B_i$. Let $C_{\max} = \max_i E_i$. The compatibility graph is the path $G_c = ([n], \{\{i, i{+}1\} : i \in [n{-}1]\})$.

6.1 Thread segments on a path

Given a schedule, we define the thread segments as follows- two neighbours $k, k{+}1$ *overlap* if some block of one is executed during the idle window of the other. Let H be the graph on $[n]$ having $\{k, k{+}1\} \in E(H)$ iff k and $k{+}1$ overlap. A *thread segment* is any interval $[i, j] \subseteq [n]$ that is a connected component of H.

▶ **Lemma 14.** *For any feasible schedule on a path, $[n]$ can be partitioned into disjoint thread segments $[i_1, j_1], \ldots, [i_t, j_t]$ that cover $[n]$, are ordered $1 = i_1 \le j_1 < i_2 \le j_2 < \cdots < i_t \le j_t = n$, and admit no overlap across different intervals.*

Proof. Overlaps can only occur on edges of the path, so H is a subgraph of 1–2–$\cdots$–n (i.e a path on n vertices); hence each component is an interval and the stated ordering follows. Any cross-interval overlap would add an edge between two components, a contradiction. ◀

By Lemma 14, it suffices to compute the optimum on a single segment $[\ell, r]$ and then concatenate segments by a prefix DP.

Orientation signature.

For each $\{k, k{+}1\}$ with $\ell \le k < r$ define (figure 14)

$$\sigma_k = \begin{cases} \rightarrow & \text{if only the first block of } k{+}1 \text{ lies in } I_k \text{ and only the second block of } k \text{ lies in } I_{k+1}, \\ \leftarrow & \text{if only the first block of } k \text{ lies in } I_{k+1} \text{ and only the second block of } k+1 \text{ lies in } I_k. \end{cases}$$

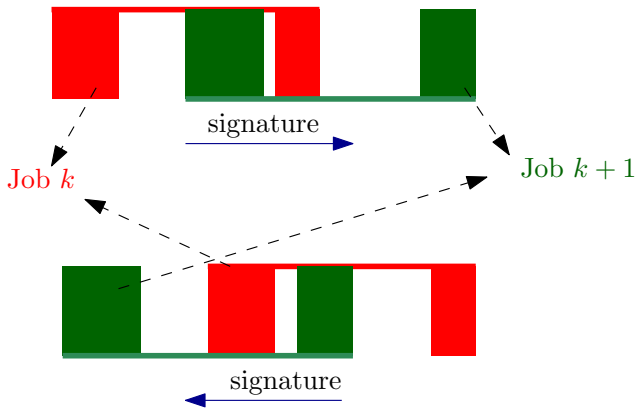


Figure 14 Different orientation signatures

Consequently, the orientation signature for a pair where one job is completely contained in the other is not defined. We now show that every pair of consecutive jobs in the same thread segment has the same orientation signature (except the first and the last pair in the sequence). But before this, we make a further simplifying observation (that also proves our claim).

▶ **Lemma 15.** *Let $[i, j]$ $(j \ge i + 2)$ be a thread segment whose compatibility graph is a path, and suppose that the jobs in $[i, j]$ are scheduled feasibly such that every job overlaps with each of its neighbors. Let π be the (increasing) ordering of the jobs in $[i, j]$ according to the starting times of their first blocks. Then π must be one of the following sequences:*

$$(i, i+1, i+2, \ldots, j), \quad (i+1, i, i+2, \ldots, j),$$
$$(j, j-1, j-2, \ldots, i), \quad (j-1, j, j-2, \ldots, i),$$

Moreover, the only jobs that can be entirely contained in the idle time of another job are i inside the idle time of $i + 1$, or j inside the idle time of $j - 1$. For the thread segments with only 2 elements, the possibilities are, trivially (i, j) and (j, i), finally the thread segment containing a single element i, $\pi = (i)$ is the only possibility.

Proof. We first show that the first job to start must belong to the set $\{i, i + 1, j - 1, j\}$. Assume for contradiction that some job $k \in \{i + 2, \ldots, j - 2\}$ starts first. Since the schedule is feasible and every job overlaps with its neighbors, job k must overlap with both $k - 1$ and $k+1$. Neither $k-1$ nor $k+1$ can be fully contained in the idle time of k, because $k-1$ must also overlap with $k - 2$ and $k + 1$ must overlap with $k + 2$. However, as the compatibility graph is a path, jobs $k - 2$ and $k + 2$ are not adjacent to k, and hence cannot overlap with k. This yields a contradiction as it forces both $k + 1$ and $k - 1$ to overlap with k from the same side, thereby overlapping among themselves as well. Therefore the first job must be one of $\{i, i + 1, j - 1, j\}$.

Next, observe that the only jobs that may be fully contained in the idle time of another job are the end jobs i and j. Indeed, any internal job $k \in [i+1, j-1]$ must overlap with two neighbors, and since these neighbors are not adjacent to each other in the path, placing k entirely inside the idle time of one neighbor would prevent it from overlapping with the other. Thus, only i may be contained in the idle time of $i+1$, and only j may be contained in the idle time of $j-1$.

We now consider the possible cases.

- **Case 1:** Suppose the schedule starts with job i. Since $i+1$ must overlap with i, the first block of $i+1$ must start in the idle time of i. Inductively, for each $k = i+2, \ldots, j-1$, the first block of k must start in the idle time of $k-1$. For the last job j, there are exactly two possibilities: either the entire job j is contained in the idle time of $j-1$, or only its first block is placed there. Either case, the job j starts after the job $j-1$ This yields the first sequences listed in the statement.
- **Case 2:** If the schedule starts with $i+1$, then job i must be entirely contained in the idle time of $i+1$. This is because, placing only the first block of i in the idle time of $i+1$ would force both i and $i+2$ to overlap, creating an overlap between i and $i+2$, which is forbidden since they are not adjacent in the compatibility graph. The remainder of the schedule then proceeds as before, again yielding the second possible sequences.

The cases where the schedule starts with j or $j-1$ are symmetric and give rise to the remaining two sequences. No other ordering is possible, completing the proof. ◀

It is not hard to see how this proof also establishes our original claim, i.e.,

▶ **Lemma 16.** *Let $[i, j]$ be any thread segment in a feasible schedule. Then, for all $k \in \{i+1, \ldots, j-2\}$, the orientation signature of the pair $\{k, k+1\}$ is identical and is either $\rightarrow$ or $\leftarrow$. Moreover, for the boundary pairs $\{i, i+1\}$ and $\{j-1, j\}$, the orientation signature is either the same as that of the internal pairs, or the first job i is fully contained in the idle time of job $i+1$, and/or the last job j is fully contained in the idle time of job $j-1$.*

In particular, the sequences $\{i, i+1, i+2, \cdots, j\}$ or $\{i+1, i, i+2, \cdots, j\}$ have $\rightarrow$ orientation signature for consecutive pairs (except possibly the first and the last pairs, i.e $\{i, i+1\}$ and $\{j-1, j\}$) and the sequences- $\{j, j-1, j-2, \cdots, i\}$ or $\{j-1, j, j-2, \cdots, i\}$ have $\leftarrow$ orientation signature.

By Lemma 15 and 16, any feasible schedule of the jobs in $[i, j]$ in which every job overlaps with its neighbors must induce one of the finitely many sequences listed above. Hence, the minimum possible makespan for $[i, j]$ is obtained by taking the minimum over all feasible schedules whose induced sequence is one of these sequences.

We now describe how these apparently different sequences are not too different from each other. Broadly speaking, there are two apparent types of sequences:

- **Case 1:** sequences in which neither endpoint job is fully contained in the idle time of its neighbor, and
- **Case 2:** sequences in which at least one of the endpoint jobs is fully contained in the idle time of its neighbor.

Below we describe a simple procedure that helps us transform any sequence in Case 2 to a sequence in Case 1 without altering the answer of an optimal schedule.

Transformation. Suppose job i is fully contained in the idle time of job $i+1$ and the sequence π begins with $(i+1, i, \ldots)$. In this case, job i can be merged into job $i+1$ by increasing the length of the first block of $i+1$ to

$$A'_{i+1} \;=\; A_{i+1} + A_i + L_i + B_i.$$

Further the idle time of job $i+1$ is reduced by exactly $A_i + L_i + B_i$. After this modification, we remove job i from consideration and treat the sequence as starting from $i+1$ with the updated processing times. This modification is valid because there is no left overlapping job for $i+1$, and we consider the case where job i is scheduled entirely within the idle time of $i+1$. Since i cannot overlap with $i+2$, but the first block of $i+2$ and the entire job i has to be scheduled in the idle time of $i+1$, i should end before the start of $i+2$. Consequently, job i can be positioned at the extreme left of the idle time of $i+1$, effectively merging with the first block of $i+1$. The other modifications when j is contained in $j-1$ follows a similar argument.

Similarly, if the sequence begins with $(j-1, j, \dots)$ and job j is fully contained in the idle time of $j-1$, we update $A'_{j-1} \;=\; A_{j-1} + A_j + L_j + B_j$, remove job j, and reduce the idle time of $j-1$ accordingly. An analogous procedure applies when the sequence ends with $(\dots, i+1, i)$ or $(\dots, j-1, j)$ with the last starting job being scheduled inside the idle time of the just previously starting job. If job i is fully contained in the idle time of $i+1$, we update the second block of $i+1$ to $B'_{i+1} \;=\; B_{i+1} + A_i + L_i + B_i$, remove job i, and reduce the idle time of $i+1$ accordingly. The case involving job j being scheduled inside the idle time of job $j-1$ is symmetric.

If none of the above transformations is possible when required by the sequence π, then π is infeasible. And, after exhaustively applying these reductions, we obtain a reduced instance in which neither end job is contained in the idle time of its neighbor, i.e *Case 1.* For such a transformed/reduced sequence, we know show a greedy algorithm to compute its minimum possible makespan.

▶ **Lemma 17.** *Let $[i, j]$ be a set of jobs whose compatibility graph is a path with edges $\{k, k+1\}$ for all $k \in \{i, \dots, j-1\}$. Suppose the jobs are required to be scheduled so that every internal job $k \in \{i+1, \dots, j-1\}$ overlaps with both of its neighbors $k-1$ and $k+1$. Given any fixed ordering of the jobs according to the starting times of their first blocks, the minimum-makespan feasible schedule (if one exists) can be computed in polynomial time. Consequently, the minimum makespan over all feasible schedules of the jobs in $[i, j]$ can be computed in polynomial time ($\mathcal{O}(n)$).*

Proof. Armed with the previous observations and explanations, it is enough for us to show that given a sequence $\{i, i+1, \cdots, j\}$ of jobs such that for each $k \in \{i, i+1, \cdots, j-1\}$, (only) the second block of k is contained in the idle time of $k+1$, the minimum makespan schedule can be computed in polynomial time.

We schedule the jobs greedily, placing each job as early as possible while maintaining feasibility for the remaining jobs. We begin by scheduling job i at time 0. Next, we schedule the first block of job $i+1$ within the idle time of job i, choosing the earliest possible start time after $S_i + A_i$ such that the remaining idle time (of $i+1$) after the completion of the second block of job i is at least A_{i+2}. The first block of job $i+2$ is then placed in this remaining idle interval.

This procedure is repeated for all subsequent jobs up to job $j-1$. More precisely, suppose that jobs up to the $k-1$-th job have already been scheduled, where $k-1 \geq i+1$. We place the first block of job k as early as possible within the idle time of job $k-1$, while ensuring that the remaining idle time (in I_k) is sufficient to accommodate the first block of job $k+1$.

Formally, job k can start in one of two ways. In the first case, there is sufficient idle time for job k to start immediately after the completion of job $k-2$, that is, at time $S_{k-2} + A_{k-2} + L_{k-2} + B_{k-2}$ (inside the idle time of $k-1$). In the second case, the idle time

of job k is not long enough, and job k must be positioned so that the idle time remaining in I_k after the completion of job $k-1$ is exactly sufficient to place the first block of job $k+1$. In this case, job k starts at time $S_{k-1} + A_{k-1} + L_{k-1} + B_{k-1} - (A_k + L_k - A_{k+1})$.

Since the idle time (of the job k) remaining after the completion of job $k-1$ must be at least A_{k+1}, job k cannot start earlier than either of the two times above. Therefore, the start time of job k is given by

$$S_k = \max\Big\{S_{k-2} + A_{k-2} + L_{k-2} + B_{k-2},\ S_{k-1} + A_{k-1} + L_{k-1} + B_{k-1} - (A_k + L_k - A_{k+1})\Big\}.$$

Finally, consider the last job j. Since there is no subsequent job, there is no requirement that its idle time must accommodate the first block of another job. Hence, job j can be scheduled as early as possible, subject to two constraints. First, it must start no earlier than the completion time of job $j-2$, namely, $S_{j-2} + A_{j-2} + L_{j-2} + B_{j-2}$. Second, its start time must allow the second block of job j to be processed after the completion of the second block of job $j-1$, which implies that it must start no earlier than $S_{j-1}+A_{j-1}+L_{j-1}+B_{j-1}-(L_j+A_j)$. Consequently, to minimize the makespan, the start time of job j is

$$S_j = \max\Big\{S_{j-2} + A_{j-2} + L_{j-2} + B_{j-2},\ S_{j-1} + A_{j-1} + L_{j-1} + B_{j-1} - (L_j + A_j)\Big\}.$$

Algorithm 1 Compute an Optimal Schedule for Jobs on a Path

Input: Jobs $i, i+1, \ldots, j$ with parameters (A_k, L_k, B_k)
Output: the optimum makespan: $\mathrm{opt}(i, j)$

Set $S_i \leftarrow 0$;
Set $S_{i+1} \leftarrow \max\{S_i + A_i + L_i + B_i - (A_{i+1} + L_{i+1} - A_{i+2}), S_i + A_i\}$
for $k = i+2$ **to** j **do**
  if $k < j$ **then**
    `// we try the earliest feasible placement`
    Set $S_k \leftarrow \max\{S_{k-1} + A_{k-1} + L_{k-1} + B_{k-1} - (A_k + L_k - A_{k+1}), S_{k-2} + A_{k-2} + L_{k-2} + B_{k-2}\}$
  else
    `// we place the final job as early as possible`
    Set $S_k \leftarrow \max\{S_{j-2} + A_{j-2} + L_{j-2} + B_{j-2}, S_{j-1} + A_{j-1} + L_{j-1} + B_{j-1} - L_j - A_j\}$;
if *there is an overlap between the block processing times* **then**
  return ∞
else
  return $S_j + A_j + L_j + B_j$

Let S_k denote the starting time of job k in the schedule produced by our procedure. We prove that for every $k \in [i, j]$, job k starts as early as possible among all feasible schedules whose starting-time order is $(i, i+1, \ldots, j)$ and in which every k overlaps with $k+1$.

Assume for contradiction that there exists another feasible schedule with start times $S'_i < S'_{i+1} < \cdots < S'_j$ and a smallest index $k \in [i, j-1]$ such that $S'_k < S_k$. By minimality of k, we have $S'_\ell \geq S_\ell$ for all $\ell \in [i, k-1]$. Since job k must overlap with job $k-1$ (and $k+1$), only the first block of k must be placed inside the idle time of $k-1$. Moreover, job k cannot start before job $k-2$ finishes, and hence

$$S'_k \ \geq\ S'_{k-2} + A_{k-2} + L_{k-2} + B_{k-2}.$$

In addition, to allow job $k+1$ to overlap with k, sufficient idle time must remain after the second block of $k-1$. This implies the bound

$$S'_k \;\geq\; S'_{k-1} + A_{k-1} + L_{k-1} + B_{k-1} - (A_k + L_k - A_{k+1}).$$

Combining these constraints gives

$$S'_k \;\geq\; \max\{S'_{k-2} + A_{k-2} + L_{k-2} + B_{k-2},\; S'_{k-1} + A_{k-1} + L_{k-1} + B_{k-1} - (A_k + L_k - A_{k+1})\}.$$

By the inductive assumption $S'_{k-2} \geq S_{k-2}$ and $S'_{k-1} \geq S_{k-1}$, and by the definition of our construction, the right-hand side is at least S_k. Hence $S'_k \geq S_k$, contradicting the assumption that $S'_k < S_k$. (the case when $k = j$ is argued similarly)

Therefore every job starts as early as possible in our schedule. It follows that the constructed schedule has minimum makespan among all feasible schedules respecting the sequence. Furthermore, for every job, we take the max over two possibilities, therefore the running time is linear in the number of jobs, i.e. $\mathcal{O}(n)$. ◀

6.2 Dynamic programming over the path

We now utilize Lemma 17 to design the required algorithm for the entirety of the path compatibility graph ,i.e., we show that knowing answer to the at most n^2 many sequences is sufficient for us to design a prefix DP algorithm that can combine these sequences effectively to compute the makespan of an optimal schedule.

Optimal schedule: minimum makespan of over all possible thread segments/sequences. Recall that the compatibility graph is the path $G_c = ([n], \{\{1,2\}, \ldots, \{n-1, n\}\})$. By Lemma 14, for any feasible schedule on $[n]$ the overlap graph H decomposes $[n]$ into disjoint thread segments $[i_1, j_1], \ldots, [i_t, j_t]$ that cover $[n]$ in order, with $1 = i_1 \leq j_1 < i_2 \leq j_2 < \cdots < i_t \leq j_t = n$, and admit no overlap across different intervals. Equivalently, jobs from distinct thread segments never run inside each other's idle windows in any schedule.

Conversely, given any collection of disjoint intervals $[i_1, j_1], \ldots, [i_t, j_t]$ with $1 = i_1 \leq j_1 < i_2 \leq \cdots \leq j_t = n$ and feasible schedules on each $[i_s, j_s]$, we can obtain a feasible schedule on $[n]$ by concatenating the segment schedules in time, placing each segment schedule so that it starts after the previous segment completes. Since there is no allowed overlap across different segments, such concatenation never destroys feasibility.

Thus the global optimum schedule is obtained by choosing a best possible partition of $[n]$ into intervals and then scheduling each interval optimally. This can be done by the following prefix DP algorithm.

Prefix DP. For $t \in \{0, 1, \ldots, n\}$, let $DP[t]$ denote the minimum attainable makespan on the prefix $[1, t]$. We set $DP[0] = 0$. For $t \geq 1$ we have the recurrence

$$DP[t] \;=\; \min_{1 \leq i \leq t}\{\, DP[i-1] + \mathrm{OPT}(i, t) \,\},$$

where $\mathrm{OPT}(i, t)$ is the minimum makespan for the interval $[i, t]$, computed by Lemma 17. Intuitively, suppose the last thread segment in an optimal schedule for $[1, t]$ is the interval $[i, t]$. Then $DP[i-1]$ already contains the optimal makespan for the preceding prefix, which we can directly reuse.

Correctness and Runtime. By Lemma 14, if $[i_1, j_1], \ldots, [i_s, j_s]$ is the thread-segment decomposition of an optimal schedule on $[n]$, then the value

$$DP[n] \;\leq\; \textstyle\sum_{k=1}^{s} \mathrm{OPT}(i_k, j_k)$$

is achieved by choosing $i = i_1$ at $t = j_1$, then $i = i_2$ at $t = j_2$, and so on. On the other hand, $DP[n]$ always corresponds to a partition of $[n]$ into intervals (thread segments) such that scheduling these segments contiguously yields a feasible schedule with makespan $DP[n]$. Therefore, for the optimal thread-segment decomposition $[i_1, j_1], \ldots, [i_s, j_s]$, we have

$$\sum_{k=1}^{s} \mathrm{OPT}(i_k, j_k) \;\leq\; DP[n].$$

Combining both inequalities, we conclude that $DP[n]$ correctly computes the makespan of an optimal schedule.

It remains to bound the running time. There are $\mathcal{O}(n^2)$ intervals $[i, t]$ with $1 \leq i \leq t \leq n$. For each interval $[i, t]$, computing $\mathrm{OPT}(i, t)$ by lemma 17 costs $\mathcal{O}(n)$ time. Summing over all intervals yields

$$\sum_{i=1}^{n} \sum_{t=i}^{n} \mathcal{O}(n) \;=\; \mathcal{O}(n^3).$$

Hence the overall running time is $\mathcal{O}(n^3)$ and our algorithm is polynomial *only* in n without any dependence on the block and idle times oj jobs.

Theorem 2. *Given a path compatibility graph G_c with n jobs, CTS can be solved in time $\mathcal{O}(n^3)$.*

7 CTS on Graphs with $\Delta(G_c) \leq 2$ (explained)

Note that the dynamic program in Section 6 solves CTS exactly on path compatibility graphs in polynomial time, without any boundedness assumptions on block or idle times. This naturally raises the question of whether such tractability extends to more general graph classes. A natural structural parameter to consider is the maximum degree $\Delta(G_c)$ of the compatibility graph. Unfortunately, CTS admits no polynomial-time algorithm when $\Delta(G_c) \geq 3$ [19], even when block and idle times are polynomially bounded (or even constant). In this section, we complement this hardness result by designing a polynomial-time algorithm for graphs with $\Delta(G_c) \leq 2$, thereby establishing a sharp complexity boundary for CTS with respect to the maximum degree. Towards, this we start by proving the following structural lemma.

▶ **Lemma 18.** *Let G_c be a cycle on jobs $[n] = \{1, \ldots, n\}$ with $n > 3$. In any feasible single-machine schedule that respects G_c (i.e., two jobs overlap only if they are adjacent on the cycle), there exists an index i such that job i does not overlap its neighbour $i+1$ (1 if $i = n$).*

Proof. For each job i we write

$$FB_i = [S_i,\, S_i + A_i], \quad I_i = [S_i + A_i,\, S_i + A_i + L_i], \quad SB_i = [S_i + A_i + L_i,\, S_i + A_i + L_i + B_i],$$

for its first block, idle interval, and second block, respectively. At any time at most one block is processed on the machine, and we say that j overlaps i if some portion of j is processed during I_i (or vice versa).

Assume for contradiction that there exists a feasible schedule in which every job i overlaps both neighbours $i - 1$ and $i + 1$ (thereby respecting the compatibility constraints).

Observation 1: If two idle intervals I_p and I_q have a positive-length intersection, then p and q overlap. Indeed, assume without loss of generality that $S_p \leq S_q$. Since blocks cannot overlap, $S_q \geq S_p + A_p$, so $I_p = [a, a + L_p]$ with $a = S_p + A_p \leq S_q$. Writing

$I_q = [c, c + L_q]$ with $c = S_q + A_q$, a positive-length intersection implies $c < a + L_p$. Thus $a \leq S_q < S_q + A_q = c < a + L_p$, so the entire first block $F_q = [S_q, S_q + A_q]$ lies inside I_p, and q overlaps p.

Observation 2: We claim that no job can be completely contained in the idle interval of a neighbour. Suppose their is a job k that is completely contained in the idle time of the job j (where the cycle has in a sequence, the vertices - j, k, l i.e. k neighbors j and l). In such a case, note that if l has to overlap with k, it also has to overlap with j (as the entire job k is scheduled in the idle time of j) which in turn violates the compatibility constraints. This proves our claim that no job can be contained in the idle time of another job.

With these two observations, consider the first scheduled job in the proposed optimal sequence. Say this job is i with the neighbors $i-1$ and $i+1$ (in the compatibility graph). From observation 2, we know that both the jobs $i+1$ and $i-1$ are right overlapping jobs of i. This necessarily means that their idle times have the block B_i (second block) in them ($SB_i \in I_{i+1}$ and $SB_i \in I_{i-1}$). This suggests that there is an intersection between I_{i-1} and I_{i+1}, which with observation 1 imply that $i+1$ and $i-1$ overlap, violating the compatibility constraints. This proves the proposed lemma. ◀

▶ **Lemma 19.** *For a compatibility graph G_c (with n jobs) and maximum degree at most 2, CTS can be solved in time polynomial in n.*

Proof. Note that we solve CTS on each connected component of G_c independently, and return the sum of their makespans as the overall solution. We categorize the connected components in G_c into three types: i) paths, ii) cycles of length greater than 3, and iii) cycles of length exactly 3. For each type, we describe below how to obtain the minimum makespan in polynomial time.

Paths: We invoke our polynomial-time algorithm designed for paths in 2.

Cycles of length $k \geq 4$. By Lemma 18, for every feasible schedule that respects the cycle C there exists an edge $\{i, i+1\}$ (indices modulo k) such that jobs i and $i+1$ do *not* overlap. In particular, this holds for some optimal schedule on C. Fix such an optimal schedule $S^\star$ and let $e^\star = \{i^\star, (i^\star + 1)\}$ be an edge whose endpoints do not overlap in $S^\star$.

For any edge e of C, let $C - e$ denote the path obtained by deleting e. This is a path on the same k jobs. Let $\mathrm{OPT}(C - e)$ be the optimal makespan for CTS on this path, computed by the path algorithm of Theorem 2. Since $C - e$ has fewer edges than C, any schedule that respects $C - e$ also respects C, so

$$\mathrm{OPT}(C - e) \;\geq\; \mathrm{OPT}(C) \qquad \text{for every edge } e \text{ of } C.$$

On the other hand, by construction $S^\star$ never places a block of $i^\star$ inside the idle window of $(i^\star + 1)$ or vice versa. Thus $S^\star$ also respects the sparser compatibility graph $C - e^\star$, and hence

$$\mathrm{OPT}(C - e^\star) \;\leq\; C_{\max}(S^\star) \;=\; \mathrm{OPT}(C).$$

Combining the two inequalities yields

$$\mathrm{OPT}(C - e^\star) \;=\; \mathrm{OPT}(C).$$

Therefore, if we compute $\mathrm{OPT}(C - e)$ for every edge e of the cycle and take the minimum value, the minimum is attained at $e^\star$ and equals the optimal makespan on the original cycle. Since C has k edges and each path instance on k jobs can be solved in time $\mathcal{O}(k^3)$, the total time required to handle a cycle of length $k \geq 4$ is $\mathcal{O}(k^4)$. This can be further improved

to $\mathcal{O}(k^3)$ by observing that we can first compute the thread segment makespans of every subpath (k^2 many) and store all the values and then for each choice (k choices) of edge break of the cycle, we do a DP as in our path algortihm (with runtime k^2), giving us a total runtime of $\mathcal{O}(k^2 \cdot k + k^2 \cdot k) = \mathcal{O}(k^3)$.

Cycles of length $k = 3$. If a component consists of a 3-cycle, then it contains only three jobs. Such a component can be solved by brute force in constant time: there are only a constant many linear orders of the three jobs and a constant many choices of which of the three edges are actually used as overlaps. For each such pattern we can check feasibility (respecting the idle-gap and compatibility constraints) and compute the resulting makespan explicitly. Taking the minimum over all patterns yields an optimal schedule for the component.

◀

Theorem 4. *Given a compatibility graph with maximum degree at most* 2 *with* n *jobs, CTS can be solved in time* $\mathcal{O}(n^3)$.

References

1. Alexander A. Ageev and Alexei E. Baburin. Approximation algorithms for uet scheduling problems with exact delays. *Operations Research Letters*, 35(4):533–540, 2007. URL: https://www.sciencedirect.com/science/article/pii/S0167637706001143, doi:10.1016/j.orl.2006.09.006.
2. Dino Ahr, József Békési, Gábor Galambos, Marcus Oswald, and Gerhard Reinelt. An exact algorithm for scheduling identical coupled tasks. *Mathematical Methods of Operations Research*, 59(2):193–203, 2004. doi:10.1007/s001860300328.
3. Karim Amrouche and Mourad Boudhar. Two machines flow shop with reentrance and exact time lag. *RAIRO – Operations Research*, 50(2):223–232, 2016. doi:10.1051/ro/2015015.
4. Philippe Baptiste. A note on scheduling identical coupled tasks in logarithmic time. *Discrete Applied Mathematics*, 158(5):583–587, 2010. doi:10.1016/j.dam.2009.10.012.
5. József Békési, Gábor Galambos, Marcus Oswald, and Gerhard Reinelt. Improved analysis of an algorithm for the coupled task problem with UET jobs. *Operations Research Letters*, 37(2):93–96, 2009. doi:10.1016/j.orl.2008.11.002.
6. Stéphane Bessy and Rodolphe Giroudeau. Parameterized complexity of a coupled-task scheduling problem. *Journal of Scheduling*, 22(3):305–313, 2019. doi:10.1007/s10951-018-0581-1.
7. Jacek Błażewicz, Grzegorz Pawlak, Michał Tanaś, and Wojciech Wojciechowicz. New algorithms for coupled tasks scheduling – a survey. *RAIRO – Operations Research*, 46(4):335–353, 2012. doi:10.1051/ro/2012020.
8. Nadia Brauner, Gerd Finke, Vassilissa Lehoux-Lebacque, Chris Potts, and Jonathan Whitehead. Scheduling of coupled tasks and one-machine no-wait robotic cells. *Computers & Operations Research*, 36(2):301–307, 2009. Scheduling for Modern Manufacturing, Logistics, and Supply Chains. URL: https://www.sciencedirect.com/science/article/pii/S0305054807001980, doi:10.1016/j.cor.2007.10.003.
9. A. Condotta and N.V. Shakhlevich. Scheduling coupled-operation jobs with exact time-lags. *Discrete Applied Mathematics*, 160(16):2370–2388, 2012. URL: https://www.sciencedirect.com/science/article/pii/S0166218X12002326, doi:10.1016/j.dam.2012.05.026.
10. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 2nd edition, 2001.
11. Benoît Darties, Rodolphe Giroudeau, Jean-Claude König, and Gilles Simonin. Some complexity and approximation results for coupled-tasks scheduling problem according to topology. *RAIRO – Operations Research*, 50(4–5):781–795, 2016. doi:10.1051/ro/2016034.
12. David Fischer and Péter Györgyi. Approximation algorithms for coupled task scheduling minimizing the sum of completion times. *Annals of Operations Research*, 328(2):1387–1408, 2023. doi:10.1007/s10479-023-05322-5.
13. M. R. Garey, D. S. Johnson, and R. Endre Tarjan. The planar hamiltonian circuit problem is NP-complete. *SIAM Journal on Computing*, 5(4):704–714, 1976. doi:10.1137/0205049.
14. Mostafa Khatami, Daniel Oron, and Amir Salehipour. Scheduling coupled tasks on parallel identical machines. *Optimization Letters*, 18(4):991–1003, 2024. doi:10.1007/s11590-023-02014-8.
15. Mostafa Khatami, Amir Salehipour, and T.C.E. Cheng. Coupled task scheduling with exact delays: Literature review and models. *European Journal of Operational Research*, 282(1):19–39, 2020. URL: https://www.sciencedirect.com/science/article/pii/S0377221719307155, doi:10.1016/j.ejor.2019.08.045.
16. A.J. Orman, C.N. Potts, A.K. Shahani, and A.R. Moore. Scheduling for a multifunction phased array radar system. *European Journal of Operational Research*, 90(1):13–25, 1996. URL: https://www.sciencedirect.com/science/article/pii/037722179500307X, doi:10.1016/0377-2217(95)00307-X.
17. Alex J. Orman and Chris N. Potts. On the complexity of coupled-task scheduling. *Discrete Applied Mathematics*, 72(1–2):141–154, 1997. doi:10.1016/S0166-218X(96)00041-8.

18 Roy D. Shapiro. Scheduling coupled tasks. *Naval Research Logistics Quarterly*, 27(3):489–498, 1980. doi:10.1002/nav.3800270312.

19 Gilles Simonin, Benoit Darties, Rodolphe Giroudeau, and Jean-Claude König. Isomorphic coupled-task scheduling problem with compatibility constraints on a single processor. *Journal of Scheduling*, 14:501–509, 2011. doi:10.1007/s10951-010-0193-x.

20 Gilles Simonin, Rodolphe Giroudeau, Benoit Darties, and Jean-Claude König. Theoretical aspects of scheduling coupled-tasks in the presence of compatibility graph. *Algorithmic Operations Research*, 7(1):1–12, 2012. URL: https://journals.lib.unb.ca/index.php/AOR/article/view/18602.

21 Wenci Yu, Han Hoogeveen, and Jan Karel Lenstra. Minimizing makespan in a two-machine flow shop with delays and unit-time operations is NP-hard. *Journal of Scheduling*, 7(5):333–348, 2004. doi:10.1023/B:JOSH.0000036858.59787.c2.